

AI-POWERED MULTI-BRAND E-BIKE RECOMMENDATION AND SMART MONITORING SYSTEM

MINI PROJECT REPORT

Submitted by

Soundarya M (24619160)

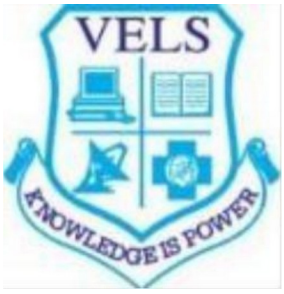
Saraniya A (24619128)

Josphin Sheran J (24619116)

Under the guidance of Dr. A. Manikandan, Associate Professor
Department of Artificial Intelligence and Machine Learning

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE -
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System" is the bonafide work of **Soundarya M (24619160), Saraniya A (24619128), Josphin Sheran J (24619116)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

SIGNATURE OF THE SUPERVISOR
Dr. A. Manikandan
Associate Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on _____ at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project work entitled “AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System” submitted to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning), is a record of original work done by us under the supervision of Dr. A. Manikandan, Associate Professor, Department of Artificial Intelligence and Machine Learning. We further declare that this report has not been submitted previously, in part or in full, for the award of any degree, diploma, associateship or fellowship at this or any other university or institution.

SOUNDARYA M 24619160
SARANIYA A 24619128
JOSPHIN SHERAN J 24619116

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of the FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH, and it is with great pleasure that we take this opportunity to thank him. We are grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our INTERNAL GUIDE, **Dr. A. Manikandan, Associate Professor, Department of AI & ML**, who has patiently guided us and provided invaluable advice and help. We would like to express our greatest gratitude to our staff members, the librarian and the non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not least, we wish to thank our parents and friends for their individual support and interest, which inspired and encouraged us to go our own way; without them we would be unable to complete this project.

SOUNDARYA M 24619160

SARANIYA A 24619128

JOSPHIN SHERAN J 24619116

TABLE OF CONTENTS

ABSTRACT **7**
LIST OF FIGURES **8**
LIST OF TABLES **9**
LIST OF ABBREVIATIONS **10**
CHAPTER 1 INTRODUCTION **11**
1.1 General 11
1.2 Background and Motivation 11
1.3 Problem Description 12
1.4 Existing System 12
1.5 Limitations of the Existing System 12
1.6 Proposed System 13
1.7 Objectives 13
1.8 Scope of the Project 14
1.9 Contributions 14
1.10 Modules 14
1.11 Organization of the Report 15
1.12 Summary 15
CHAPTER 2 LITERATURE REVIEW **16**
2.1 Recommender Systems and Product Ranking 16
2.2 Multi-criteria Decision Making 16
2.3 Explainable Recommendation 17
2.4 Recommendation in the E-bike and Micromobility Domain 17
2.5 IoT Sensing on E-bikes 17
2.6 Battery and Vehicle Condition Monitoring 17
2.7 Sensors Used 17
2.8 Comparison of Selected Studies 18
2.9 Technology Background 18
2.10 Existing versus Proposed Approach 19
2.11 Research Gap 20
2.12 Summary 20
CHAPTER 3 SYSTEM DESIGN **21**
3.1 Existing System 21
3.2 Proposed System 21
3.3 System Architecture 22
3.4 Overall Workflow and UML Diagrams 23
3.5 Algorithms 26
3.6 Data Flow 26
3.7 System Modules 27
3.8 Hardware Connection Design 28
3.9 Hardware Requirements 29
3.10 Software Requirements 30
3.11 Database Design 30
3.12 Component Description 32

3.13 User Interface Design....32
3.14 Feasibility Study....32
3.15 Requirement Specification....33
3.16 Summary....34
CHAPTER 4 IMPLEMENTATION AND METHODOLOGY....35
4.1 Methodology....35
4.2 Implementation....35
4.3 Working Process of the System....47
4.4 Summary....50
CHAPTER 5 RESULTS AND DISCUSSION....51
5.1 Experimental Setup....51
5.2 Recommendation Results....51
5.3 Dashboard Results....53
5.4 Monitoring Evidence....53
5.5 Implementation Status Summary....53
5.6 Testing and Evidence....53
5.7 Observations on the Current Implementation....54
5.8 Overall Discussion....55
5.9 Summary....55
CHAPTER 6 LIVE MONITORING TOOL AND DEMONSTRATION....56
6.1 Overview....56
6.2 Live Monitoring Dashboard....56
6.3 Telemetry Demonstration....58
6.4 Application Areas....60
6.5 Alerts and Telegram Messages....61
6.6 Summary....61
CHAPTER 7 CONCLUSION AND FUTURE WORK....62
7.1 Conclusion....62
7.2 Achievement of Project Objectives....62
7.3 Limitations....63
7.4 Future Scope....63
7.5 Summary....63
APPENDIX....65
9.1 Telemetry and Configuration Variables....65
9.2 Alert Rule Conditions....65
SOURCE CODE....66
10.1 Overview of the Source Code....66
10.2 ESP32 Firmware (excerpts)....66
10.3 Website Recommendation and Bridge Logic (excerpts)....69
10.4 Complete Supabase Bridge Script....70
CONFERENCE PARTICIPATION CERTIFICATES....74

ABSTRACT

Buying an electric two-wheeler is difficult for a first-time buyer because the market contains many brands and models whose specifications (battery type, motor rating, claimed range, top speed, price) are hard to relate to personal needs. Once the vehicle is bought, its battery, motor and frame are rarely monitored, so overheating, abnormal vibration or unauthorised movement may go unnoticed. This project presents **VoltWise**, a single web platform that joins these two stages: pre-purchase recommendation and post-purchase monitoring.

The recommendation part is a website that lists eight electric scooter models from several brands, allows search, category filtering, sorting, side-by-side comparison of up to three models and a shortlist. A five-question finder (primary use, budget, required range, terrain and riding priority) ranks the models with a transparent **rule-based weighted score** (use 35%, budget 25%, range 15%, terrain 15%, priority 10%) and shows the top three with a match percentage and reasons. The score is not produced by a trained machine-learning model.

The monitoring part is an **ESP32-based prototype** fitted with two DS18B20 temperature sensors (motor and battery), an MPU6050 accelerometer/gyroscope for vibration, tilt and impact detection, a BMP280 sensor, a 0.96-inch OLED display, a buzzer and red/green LEDs. The firmware applies confirmation and hysteresis to its thresholds, classifies the state as NORMAL, WARNING or CRITICAL, raises local audible and visual alarms, uploads telemetry to a Supabase database every three seconds, and sends Telegram messages for warnings and critical events. The website dashboard polls the database and switches between live hardware data and a clearly labelled demonstration simulator. Voltage and current sensing (INA219), GPS and solar charging are planned but are not part of the implemented prototype.

The report describes the design, implementation and evidence available at the time of writing, and separates implemented, simulated and future features.

Keywords: e-bike recommendation, electric scooter, multi-criteria scoring, explainable recommendation, ESP32, IoT monitoring, DS18B20, MPU6050, Supabase, vehicle health alerts.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	Overall architecture of the VoltWise platform	23
Figure 3.2	Use case diagram of the platform	24
Figure 3.3	Sequence diagram of telemetry, alert and dashboard polling	25
Figure 3.4	State diagram of the vehicle status	25
Figure 3.5	Level-0 data flow diagram (context diagram)	26
Figure 3.6	Level-1 data flow diagram	27
Figure 3.7	Functional modules of the platform	27
Figure 3.8	Hardware connection diagram of the ESP32 sensing node	29
Figure 3.9	Tables used by the dashboard	31
Figure 4.1	VoltWise home page (desktop view rendered from the supplied source file)	36
Figure 4.2	Catalogue with search, sort and category filters on a phone (screenshot of the deployed site)	37
Figure 4.3	Recommendation finder, step 3 of 5 (desktop view rendered from the supplied source file)	38
Figure 4.4	Workflow of the recommendation engine	39
Figure 4.5	Side-by-side comparison of three models (desktop view rendered from the supplied source file)	42
Figure 4.6	Hardware prototype: ESP32 on a breadboard with MPU6050, BMP280, OLED, buzzer, red and green LEDs and a DS18B20 probe	43
Figure 4.7	Flow of the ESP32 firmware	46
Figure 5.1	Ranked recommendation results for the sample answers	52
Figure 6.1	Monitoring dashboard on a phone with the ESP32 node offline: (a) header, (b) extended telemetry and trend chart, (c) motion panel and demonstration control	57
Figure 6.2	Simulated fleet list on the Fleet section (fictional demonstration units)	58
Figure 6.3	Dashboard in the Warning demonstration mode (simulated values, rendered from the source file)	59
Figure 6.4	Dashboard in the Critical demonstration mode (simulated values, rendered from the source file)	60

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Comparison of selected studies with this project	18
Table 2.2	Comparison of approaches	19
Table 3.1	Implemented versus planned components	22
Table 3.2	Module description and status	28
Table 3.3	Hardware components of the prototype	29
Table 3.4	Software requirements	30
Table 3.5	Telemetry table columns read by the dashboard	31
Table 3.6	Alert table columns read by the dashboard	31
Table 3.7	Roles and key properties of the components	32
Table 3.8	Functional requirements	33
Table 3.9	Non-functional requirements	33
Table 4.1	Catalogue records implemented in the website	37
Table 4.2	Finder questions and options	38
Table 4.3	Criterion scoring rules implemented in the website	40
Table 4.4	Scores produced by the engine for the worked example	40
Table 4.5	Pin and address assignment in the firmware	43
Table 4.6	Thresholds used by the firmware	44
Table 4.7	Alert codes and local responses	45
Table 4.8	Scenario A ranking	48
Table 4.9	Scenario B ranking	48
Table 4.10	Scenario C ranking	48
Table 4.11	Event-to-response mapping	50
Table 5.1	Status of features at the time of writing	53
Table 5.2	Test cases and their evidence status	54
Table 7.1	Objectives and their achievement	62
Table 9.1	Variables used by the monitoring node and dashboard	65
Table 9.2	Alert rules implemented in the firmware	65

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
IoT	Internet of Things
ESP32	Espressif 32-bit Wi-Fi/Bluetooth microcontroller
MCDM	Multi-Criteria Decision Making
SAW	Simple Additive Weighting
EV	Electric Vehicle
BMS	Battery Management System
SoC	State of Charge
SoH	State of Health
BLDC	Brushless Direct Current (motor)
I²C	Inter-Integrated Circuit bus
GPIO	General-Purpose Input/Output
OLED	Organic Light-Emitting Diode (display)
LED	Light-Emitting Diode
MPU	Motion Processing Unit
BMP	Barometric Pressure (sensor family)
DS18B20	Digital 1-Wire temperature sensor
INA219	Current/power monitor IC
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
REST	Representational State Transfer
API	Application Programming Interface
JSON	JavaScript Object Notation
UI	User Interface
DOM	Document Object Model
DFD	Data Flow Diagram
RLS	Row-Level Security
GPS	Global Positioning System
INR	Indian Rupee

CHAPTER 1
INTRODUCTION

1.1 General

Electric two-wheelers are being adopted quickly in India as a low-running-cost and low-emission alternative to petrol vehicles, and the number of brands and models available to a buyer has grown with them. A buyer must compare price, battery chemistry and voltage, motor rating, claimed range, top speed and weight, and then decide which combination suits his or her daily use. Comparison pages usually list the numbers but leave the interpretation to the buyer, and once the vehicle has been purchased there is normally no tool that tells the owner whether the battery or motor is running hotter than it should, whether the frame is vibrating abnormally, or whether the parked vehicle has been moved.

This project, **AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System**, addresses both problems in one platform called **VoltWise** ("Smart Mobility Systems"). The first stage helps a person choose a vehicle: a website presents a multi-brand catalogue and a five-question finder that ranks the catalogue with a transparent weighted score. The second stage looks after the vehicle after purchase: an ESP32-based sensing node measures motor and battery temperature, vibration, tilt and impact, raises local and remote alerts, and sends its data to a dashboard on the same website.

> **Terminology used in this report.** The project title uses the term "e-bike". The catalogue implemented in the website contains eight seated electric **scooters** from different manufacturers (Yakuza, Komaki, AMO Mobility, Gigaglitz, BGauss, NUV and OKAI). In this report "e-bike" and "electric two-wheeler" are used in the general sense of the title, and "e-scooter" is used when referring to the specific models in the catalogue.

1.2 Background and Motivation

Electric two-wheelers have moved from a niche product to an everyday choice for students, office workers and delivery riders. Manufacturers publish specifications such as battery voltage, motor power, claimed range and top speed, but these figures describe the vehicle, not the rider. A student who rides a few kilometres on city roads and a delivery rider who covers long distances in a day need different vehicles even if their budgets are the same. At the same time, lithium-based battery packs and motors can overheat, and a two-wheeler is exposed to vibration, impact and theft. These two observations motivated the idea of a single platform that first helps the buyer decide and then keeps watching the vehicle.

Low-cost microcontrollers such as the ESP32, digital temperature sensors, motion sensors and free-tier cloud services now make it possible for a student team to build such a monitoring node. A simple, transparent scoring rule is sufficient to explain a recommendation to a non-expert.

The motivation of this project is therefore to combine these accessible technologies into a working, honest and explainable prototype.

1.3 Problem Description

The existing way of buying and owning an electric two-wheeler has the following weaknesses:

- **Specifications are hard to interpret.** Non-technical buyers find terms such as “250 W BLDC”, “48 V” or “claimed range” difficult to relate to their own journeys.
- **Comparison is manual.** Brand websites show only their own models, and general comparison pages expect the buyer to match specifications to needs.
- **Recommendations are rarely explained.** When a tool does suggest a model, it often does not say why.
- **Selection and ownership are disconnected.** The tool used to choose a vehicle is separate from any tool used to monitor it.
- **Raw sensor data is not actionable.** A reading such as “58 °C” is of little use unless it is compared with limits and turned into a status and a message.

The core problem is therefore the lack of a single, understandable platform that supports an explainable purchase decision and then continues to watch the health and safety of the chosen vehicle.

1.4 Existing System

Buyers currently use a combination of separate resources: brand websites and dealerships that present only their own models; general comparison pages that show specifications side by side; basic recommendation tools that suggest models without an explanation; and aftermarket trackers or manufacturer telematics that monitor a vehicle but are not connected to the selection stage. Cross-brand comparison needs manual effort, personalisation for non-technical users is limited, and the user experience is split across several services. The existing system is described further in Chapter 3.

1.5 Limitations of the Existing System

- Cross-brand comparison needs manual effort by the buyer.
- Little personalisation for non-technical users.
- Explanations of why a model is suitable are limited or absent.
- Selection tools and monitoring tools are separate, so the user experience is fragmented.
- Raw sensor values are not always translated into a status and an understandable message.
- Demonstration data and real measurements are often not clearly separated on dashboards.

1.6 Proposed System

The proposed system, VoltWise, is a web platform with two connected stages (see Figure 3.1 in Section 3.3). In the **recommendation stage**, the rider's five answers are compared with the specifications of every catalogue model and converted into a weighted score. In the **monitoring stage**, the ESP32 node reads its sensors every 500 ms, decides the vehicle status, drives the buzzer, LEDs and OLED, and uploads data to a cloud database from which the dashboard reads. Both stages are presented by the same web front end. The main advantages are:

- The scoring method is simple, transparent and can be explained to a non-technical user; no black-box model is involved.
- The user's answers stay in the browser; no account or server-side profile is required for the recommendation.
- Local alarms (buzzer, LEDs, OLED) work even when Wi-Fi is not available.
- Alerts use confirmation counts and hysteresis, which reduces false alarms caused by a reading hovering near a limit.
- The dashboard works as a demonstration even without hardware and states plainly when data is simulated.
- Low cost: commonly available modules and free-tier cloud services are used.

1.7 Objectives

The general objective is to demonstrate that product recommendation and vehicle-health monitoring can be delivered through one web platform, using simple and transparent methods that a student prototype can build and a user can understand. The specific objectives are:

1. To store multi-brand e-scooter specifications in a structured data array and display them in a searchable, filterable and sortable catalogue.
2. To implement five scored criteria (primary use, budget, required range, terrain and riding priority) with fixed weights summing to 100%.
3. To display the top three ranked models with a match percentage, key specifications and short reasons.
4. To implement side-by-side comparison of two or three models and a shortlist stored in the browser.
5. To implement ESP32 firmware that reads two DS18B20 sensors, an MPU6050 and a BMP280 at 500 ms intervals and learns the mounting orientation at start-up.
6. To implement WARNING/CRITICAL logic with confirmation samples and hysteresis, and special CRITICAL conditions for impact and movement while parked.
7. To upload telemetry every 3 s and alerts on state change to Supabase, and to send Telegram messages for warnings and critical alerts.

8. To build a dashboard that polls Supabase, shows live values when the node is reporting, detects an offline node after 15 s without data and otherwise falls back to a labelled simulator.

1.8 Scope of the Project

- **Recommendation side:** a single-page website written in HTML, CSS and JavaScript with a catalogue of eight models, a five-step finder, ranked results with reasons, comparison of up to three models, a browser-stored shortlist and model detail views.
- **Monitoring side:** an ESP32 DevKit V1 prototype on a breadboard with two DS18B20 sensors, an MPU6050, a BMP280, an OLED, a buzzer and two LEDs; firmware with alert logic; Supabase for telemetry and alert storage; Telegram for instant messages.
- **Dashboard side:** KPI cards, trend charts, movement panel, alert centre, maintenance insights, hardware-readiness panel and a simulator for demonstration.
- **Not in scope of the current prototype:** battery voltage and current sensing (INA219 not yet fitted), GPS tracking, solar charging, installation on a real vehicle, and any trained machine-learning model.

The prototype is intended for personal electric scooters and e-bikes and for demonstration in an academic setting. Applications include guiding first-time buyers, personal vehicle monitoring (over-temperature, vibration, impact and movement alerts), small campus fleets in the future, and teaching IoT and explainable decision support.

1.9 Contributions

- A multi-brand catalogue and a transparent five-criterion weighted scoring engine with explanations.
- A low-cost ESP32 monitoring node with orientation-independent vibration and tilt estimation, confirmation and hysteresis logic, distinct buzzer patterns and deferred network alerts.
- A dashboard that reads live data from Supabase and clearly separates live, simulated and planned data.
- A single platform that joins the selection stage and the ownership stage of an electric two-wheeler.
- A documented list of implemented, simulated and planned features, with the limitations of the prototype stated openly.

1.10 Modules

The platform is divided into seven modules: (1) user input, (2) e-bike database, (3) recommendation engine, (4) explainability, (5) IoT monitoring, (6) alert and maintenance, and (7) web dashboard. Modules 1–4 form the recommendation stage and modules 5–7 form the monitoring stage. Each module is described in Chapter 3 and its implementation in Chapter 4.

1.11 Organization of the Report

- **Chapter 2 (Literature Review)** reviews recommendation, multi-criteria ranking, explainable recommendation and IoT monitoring research and identifies the research gap.
- **Chapter 3 (System Design)** presents the existing and proposed systems, architecture, workflow, modules, data flow, hardware and software requirements, database, feasibility and requirement specification.
- **Chapter 4 (Implementation and Methodology)** describes the website, the recommendation engine, the ESP32 sensing node, the communication and the working process.
- **Chapter 5 (Results and Discussion)** presents the recommendation results, the dashboard behaviour, the implementation status, testing and observations.
- **Chapter 6 (Live Monitoring Tool and Demonstration)** explains the dashboard, its data modes, the demonstration scenarios and the application areas.
- **Chapter 7 (Conclusion and Future Work)** summarises the outcomes, the achievement of the objectives, the limitations and the future scope.
- The References, Appendix, Source Code and Certificates follow the chapters.

1.12 Summary

This chapter introduced the need for an integrated selection-and-monitoring platform for electric two-wheelers, stated the problem, outlined the existing and proposed systems, listed the objectives, scope and contributions, and named the seven modules. The next chapter reviews the literature related to recommendation, multi-criteria ranking and IoT monitoring.

CHAPTER 2
LITERATURE REVIEW

This project touches four research areas: recommender systems and multi-criteria product ranking, explainable recommendation, internet-of-things (IoT) sensing for electric vehicles, and battery and condition monitoring. Each area has a large literature of its own; the review below concentrates on work that explains the methods used in this project or that is closest to its aims. Only sources whose bibliographic details could be verified are cited (Section References).

The review serves the following purposes:

- to place the rule-based, weighted scoring used in the website within the wider family of recommendation and multi-criteria decision methods;
- to understand why explanation matters in a recommendation shown to a non-expert buyer;
- to review how IoT devices built around the ESP32 and similar controllers are used to monitor e-bikes and electric vehicles;
- to identify, without exaggeration, the gap that this project tries to fill.

2.1 Recommender Systems and Product Ranking

Adomavicius and Tuzhilin [8] surveyed recommender systems and classified them into content-based, collaborative and hybrid approaches, and the Recommender Systems Handbook [9] gives a broad treatment of these and of knowledge-based methods. Collaborative methods need a history of ratings from many users, which a new product-selection website does not have. For infrequent, high-value purchases such as a vehicle, the buyer’s stated requirements are usually more informative than past behaviour, which makes requirement-driven (knowledge- or constraint-based) ranking a natural choice for a small prototype. The VoltWise finder follows this line: it asks for requirements explicitly and compares them with product attributes.

2.2 Multi-criteria Decision Making

Choosing a vehicle is a multi-criteria decision in which price, range, terrain suitability and use all matter at once. Hwang and Yoon [6] describe a family of multiple-attribute decision methods, including simple additive weighting and TOPSIS, and Saaty [7] introduced the Analytic Hierarchy Process for deriving criterion weights from pairwise judgements. Simple additive weighting multiplies each criterion score by a weight and sums the results; it is easy to compute and easy to explain. The VoltWise engine is a simple additive weighting model with fixed weights. It does not derive its weights with AHP or normalise attributes with TOPSIS, and it does not claim to be optimal; these remain possible extensions.

2.3 Explainable Recommendation

Zhang and Chen [5] review explainable recommendation and argue that explanations improve transparency, persuasiveness, trust and user satisfaction. They distinguish the model that produces a recommendation from the explanation shown to the user. In the present work the model is transparent by construction (a visible weighted sum) and the explanation is a short set of statements derived from the per-criterion scores, for example whether the model fits the stated budget or terrain.

2.4 Recommendation in the E-bike and Micromobility Domain

Yan, O'Connor and Liu [1] present U-Park, a recommendation system for shared electric micromobility services. It uses machine-learning models on trip and parking data from a shared e-bike operator to predict a user's destination and recommend a parking station, and the authors report a destination-prediction accuracy of over 97% on their data set. U-Park shows that recommendation techniques are applied to e-bikes, but its problem (where to park a shared vehicle) and its data (trip histories) differ from the problem of choosing which model to buy, and it does not involve sensors on the vehicle.

2.5 IoT Sensing on E-bikes

Flores Mendieta et al. [2] built an IoT device for measurement of e-bike energy consumption, using an ESP32, the ThingsBoard platform and an Android application, with Wi-Fi, Bluetooth and GPS communication and a custom PCB. They validated their measurements against reference instruments. Their work confirms that the ESP32 is a suitable controller for e-bike data acquisition, and it concentrates on energy and tracking performance rather than on purchase guidance. General surveys of IoT architectures and protocols [10], [11] explain the layered pattern — sensing node, network, cloud storage, application — that the present prototype also follows.

2.6 Battery and Vehicle Condition Monitoring

Umathe and Hiware [3] address battery management in electric vehicles by combining artificial intelligence with IoT, and Ali, Hazari and Mannan [4] describe an IoT-based battery management and monitoring system that notifies the user of battery condition in real time and aims to reduce hazardous situations. Both stress that battery temperature and electrical condition should be watched continuously. The present prototype measures battery and motor surface temperature and motion; it does not yet measure voltage or current, so it covers only part of what these systems monitor.

2.7 Sensors Used

The firmware relies on well-documented components: the DS18B20 digital thermometer on a 1-Wire bus [12], the MPU6050 six-axis motion sensor [13], the BMP280 pressure and

temperature sensor [14] and the ESP32 microcontroller [15]. Their data sheets give the interfaces, ranges and addresses used in Chapter 4.

2.8 Comparison of Selected Studies

Table 2.1: Comparison of selected studies with this project

Ref.	Year	Focus	Relation to this project
[1] Yan et al.	2024	ML-based parking recommendation for shared e-bikes	Shows recommendation in the e-bike domain; different task (parking) and data (trips)
[2] Flores Mendieta et al.	2024	ESP32 IoT device for e-bike data acquisition and energy analysis	Confirms ESP32 for e-bike sensing; no purchase recommendation
[3] Umathe and Hiware	2022	AI and IoT battery management for electric vehicles	Battery-focused; no product selection stage
[4] Ali et al.	2023	IoT battery management and monitoring with real-time user notification	Similar notification idea; battery-focused
[5] Zhang and Chen	2020	Survey of explainable recommendation	Supports the explanation design
[6], [7] Hwang and Yoon; Saaty	1981; 1980	Multi-attribute decision methods; AHP	Basis for weighted-score ranking

2.9 Technology Background

Layered IoT Architecture

IoT systems are commonly described in layers [10], [11]: a sensing layer that measures physical quantities, a network layer that carries the data, a data or middleware layer that stores and processes it, and an application layer that presents it to the user. In this project the sensing layer is the DS18B20, MPU6050 and BMP280 on the ESP32; the network layer is Wi-Fi with HTTPS; the data layer is Supabase; and the application layer is the VoltWise dashboard and Telegram.

1-Wire Temperature Sensing

The DS18B20 communicates over a single data line using the 1-Wire protocol and returns a digital temperature value [12]. Several sensors can share one bus because each has a unique 64-bit address; this is how the firmware reads two probes on GPIO4 and distinguishes the motor probe (index 0) from the battery probe (index 1). A pull-up resistor (4.7 kΩ in this prototype) keeps the line high when no device is pulling it low. The resolution can be set to 9–12 bits; the firmware uses 10 bits as a compromise between precision and conversion time.

Accelerometer-Based Vibration, Tilt and Impact

The MPU6050 contains a three-axis accelerometer and a three-axis gyroscope and communicates over I²C [13]. At rest the accelerometer measures gravity (about 9.81 m/s²) along the axes that point up or down. If the board moves, the measured vector changes. The

firmware therefore records a reference gravity vector at boot, computes vibration as the length of the difference between the present and reference vectors, and computes tilt as the angle between the two vectors. A very large difference is treated as an impact. This approach does not depend on the board being mounted flat.

I²C Bus

I²C uses two lines, SDA (data) and SCL (clock), and each device has an address. The MPU6050, BMP280 (0x76) and OLED (0x3C) share GPIO21 and GPIO22, which saves pins on the ESP32.

Barometric Sensor

The BMP280 measures air pressure and temperature [14]. The prototype records ambient temperature and pressure as context readings; they are not used in the alert thresholds.

ESP32 and Wi-Fi Communication

The ESP32 integrates a dual-core microcontroller with Wi-Fi and Bluetooth [15]. In this project it operates in station mode, joins a 2.4 GHz network and sends HTTPS requests to Supabase and the Telegram Bot API [16], [17]. Network calls can block for several seconds, so the firmware defers them until the buzzer pattern has finished.

REST and PostgreSQL

Supabase exposes PostgreSQL tables through a REST interface [16]. The website reads rows by sending an HTTP GET request with filters such as the device identifier, sort order and limit. This allows the browser and the ESP32 to be on different networks.

Rule-Based Scoring

A rule-based weighted score is a form of simple additive weighting [6]: criterion scores are multiplied by weights and added. Its advantages for a student project are determinism (the same inputs always give the same ranking), transparency and the absence of a need for training data.

2.10 Existing versus Proposed Approach

Table 2.2: Comparison of approaches

Feature	Brand websites	Comparison pages	IoT trackers	This project
Multi-brand catalogue	No	Yes	No	Yes (8 models)
Personalised ranking	No	Limited	No	Yes (5 criteria)
Explanation of ranking	No	Limited	No	Yes (reasons, weights)
Temperature / vibration monitoring	No	No	Some	Yes (prototype)
Local alarm	No	No	Some	Yes (buzzer, LEDs, OLED)

Feature	Brand websites	Comparison pages	IoT trackers	This project
Remote alert	No	No	Yes	Yes (Telegram, dashboard)
Selection and monitoring in one platform	No	No	No	Yes

The comparison is a qualitative summary of typical offerings and is not a survey of every product on the market.

2.11 Research Gap

The reviewed work suggests that recommendation research and vehicle-monitoring research are usually reported separately: recommenders for micromobility concentrate on routes, destinations or parking, and IoT monitoring papers concentrate on measurement, tracking or battery management. This review does not claim that no integrated system exists anywhere; it only observes that, among the sources reviewed, the combination of (a) an explainable, requirement-based model ranking for a buyer and (b) a low-cost ESP32 monitoring node feeding a dashboard in the same web application was not found. The gaps addressed by this project are:

- **Separation of selection and ownership:** the tool that helps choose a vehicle is not connected to the tool that watches it.
- **Limited explanation for non-experts:** specification lists are shown without a reason for the ranking.
- **Unclear data provenance in dashboards:** demonstration values and live values are often mixed; this project labels them explicitly.
- **Low-cost prototypes:** many solutions assume dedicated hardware; this one is built from common modules and free-tier services.

2.12 Summary

The literature supports three design choices: a transparent weighted-score model for a small catalogue, short explanations next to every recommendation, and an ESP32 sensing node that reports to a cloud service. It also shows what the prototype lacks: electrical measurements, field validation and learning-based personalisation. These points are carried into the system design and the future work.

> **Note on references.** Some papers named in the project presentation could not be verified from their titles and years alone and are therefore not cited here (see the list of items for confirmation supplied with this report). They can be added once the team provides full bibliographic details or links.

CHAPTER 3
SYSTEM DESIGN

3.1 Existing System

Overview

Buyers of electric two-wheelers currently use a combination of separate resources:

1. **Brand websites and dealerships** that present only their own models.
2. **General comparison and listing pages** that show specifications side by side and leave the interpretation to the buyer.
3. **Basic recommendation tools** that suggest models but often without an explanation.
4. **Manufacturer or aftermarket telematics and trackers**, which monitor a vehicle but are not connected to the selection stage.

Limitations of the Existing Approach

- Cross-brand comparison needs manual effort.
- Little personalisation for non-technical users.
- Explanations of why a model is suitable are limited or absent.
- Monitoring and selection tools are fragmented, so the user experience is split across several services.
- Raw sensor values are not always translated into a status and an understandable message.

3.2 Proposed System

Overview

The proposed system, VoltWise, is a web platform with two connected stages. The recommendation stage helps the user find and compare suitable models; the monitoring stage tracks the condition of the vehicle by means of an ESP32 sensing node and shows the result on a dashboard in the same site.

Features of the Proposed System

- Multi-brand catalogue with search, category filter and sorting.
- Five-question finder with a transparent weighted score and top-three results with reasons.
- Side-by-side comparison (up to three models) and a shortlist kept in the browser.
- ESP32 node with temperature, vibration, tilt, impact, ambient pressure/temperature sensing.
- Local alert outputs (buzzer, LEDs, OLED) and remote alerts (Supabase, Telegram).

- Dashboard with live-or-demo data source indicator, trend charts, alert centre and maintenance suggestions.

Implemented Components and Planned Enhancements

To avoid ambiguity, the following table separates what exists in the prototype from what is only planned.

Table 3.1: Implemented versus planned components

Component / feature	Status	Remarks
Website catalogue, finder, compare, shortlist	Implemented	Supplied source (voltwise.html); deployed on Netlify
Rule-based weighted scoring	Implemented	Five criteria, fixed weights
ESP32, 2 × DS18B20, MPU6050, BMP280, OLED, buzzer, LEDs	Implemented (hardware photo, firmware)	Breadboard prototype
Alert logic with hysteresis, buzzer patterns, OLED alert screen	Implemented in firmware	Demonstrated on the prototype
Supabase upload (3 s) and alert upload	Implemented in firmware source	Demonstrated on the prototype
Telegram messages	Implemented in firmware source	Demonstrated on the prototype
Dashboard polling of Supabase	Implemented in website source	Offline state evidenced by screenshots
Dashboard simulator and 12-unit fleet	Simulated	Clearly labelled demo data
Battery voltage / current (INA219)	Not fitted yet	Code path exists, disabled by a compile-time flag
State of charge, state of health, health score	Simulated / indicative	Not measured by hardware
NEO-6M GPS, solar panel	Planned	Components listed by the team; not implemented
Trained machine-learning model	Not implemented	Future work

Advantages

- Transparent: the user can see how a score is built.
- Modular: the scoring, the dashboard and the firmware can be changed independently.
- Honest data labelling: simulated and live data are distinguished.
- Inexpensive to build and host.

3.3 System Architecture

The architecture has four layers (see Figure 3.1): the sensors, the ESP32 sensing node, the cloud backend and the web platform.

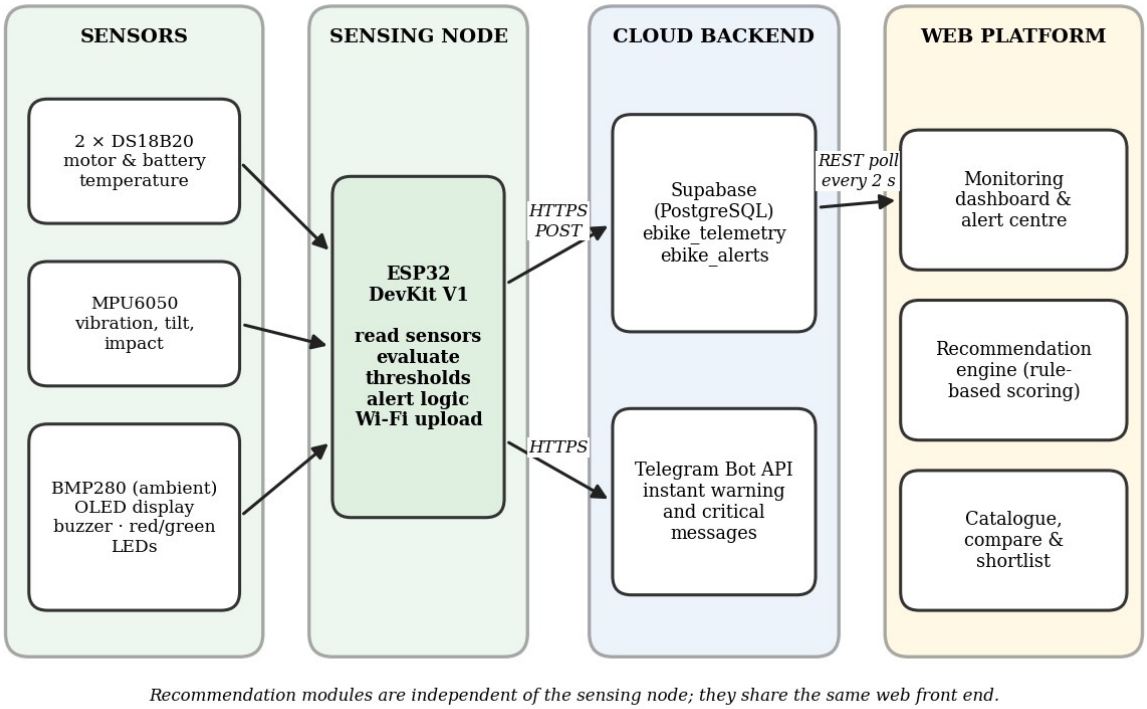


Figure 3.1: Overall architecture of the VoltWise platform

- **Sensors:** two DS18B20 probes (motor and battery), an MPU6050 on the I²C bus, a BMP280 on the I²C bus, an OLED display, a buzzer and two LEDs.
- **Sensing node:** the ESP32 reads sensors every 500 ms, evaluates thresholds, drives local outputs, and uses Wi-Fi to send data over HTTPS.
- **Cloud backend:** Supabase stores telemetry rows and alert rows; the Telegram Bot API delivers instant messages.
- **Web platform:** the single-page website contains the recommendation engine, catalogue and the monitoring dashboard, which polls Supabase every 2 s.

3.4 Overall Workflow and UML Diagrams

Use Case Diagram

The main actor is the user. The ESP32 node is an external actor that supplies telemetry and alerts (Figure 3.2).

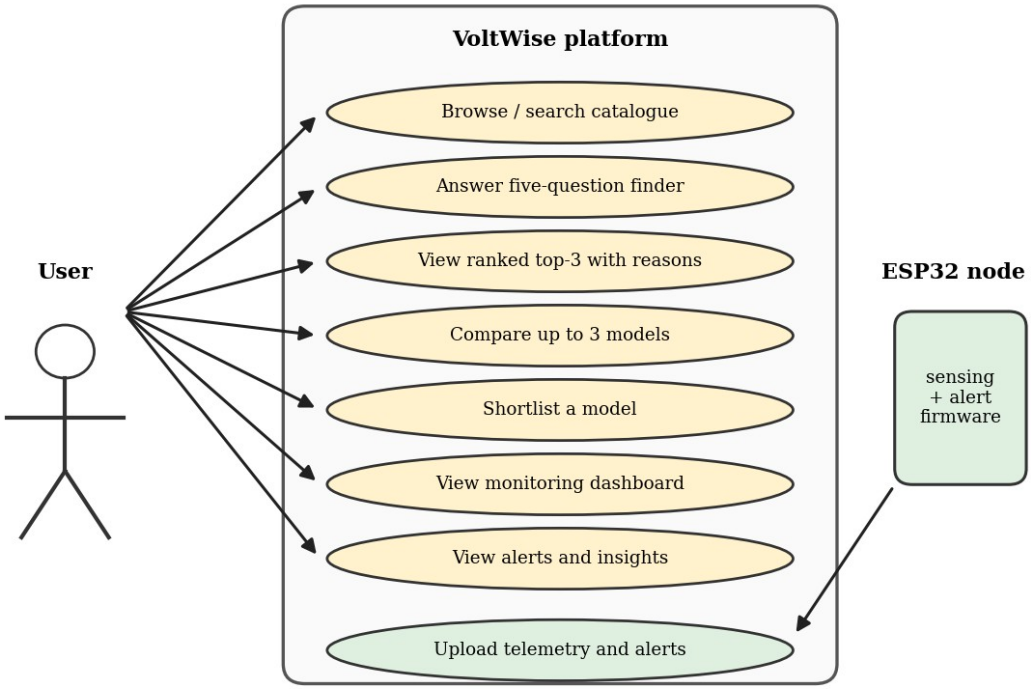


Figure 3.2: Use case diagram of the platform

Sequence Diagram of the Monitoring Process

Figure 3.3 shows the order of messages between the node, the database, the dashboard and Telegram.

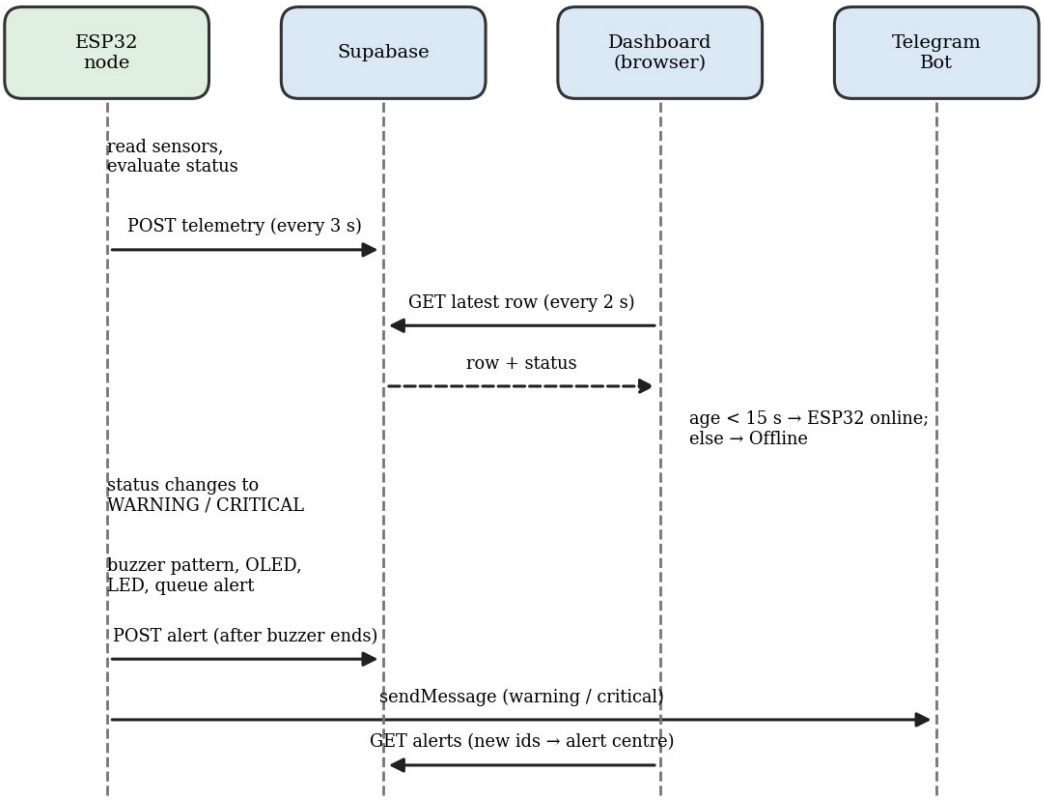


Figure 3.3: Sequence diagram of telemetry, alert and dashboard polling

State Diagram of the Vehicle Status

Figure 3.4 shows the three states and the conditions for moving between them. The conditions use confirmation samples and hysteresis as described in Section 4.2.7.

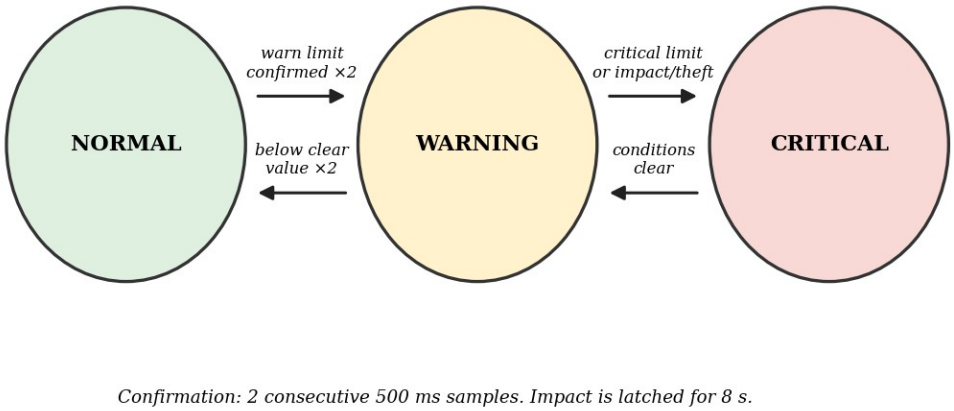


Figure 3.4: State diagram of the vehicle status

3.5 Algorithms

Recommendation Algorithm

```
Input: answers A = (use, budget, range, terrain, priority); catalogue B of 8 models
Output: ranked list of models
1  for each model b in B:
2      S_use      <- scoreUse(b, A.use)
3      S_budget   <- scoreBudget(b, A.budget)
4      S_range    <- scoreRange(b, A.range)
5      S_terrain  <- scoreTerrain(b, A.terrain)
6      S_priority <- scorePriority(b, A.priority)
7      b.score    <- 0.35*S_use + 0.25*S_budget + 0.15*S_range
                     + 0.15*S_terrain + 0.10*S_priority      (rounded to 1 decimal)
8  sort B by score descending; if scores are equal, lower price first
9  display the first three models with round(score) as "% match" and reasons
```

Monitoring Algorithm (every 500 ms)

```
1  read motorTemp, batteryTemp (DS18B20), a = (ax, ay, az) (MPU6050), pressure,
ambient (BMP280)
2  vibration <- | a - a_ref |;   tilt <- angle(a, a_ref);   impact <- vibration >
12
3  if still (vibration < 0.8) for 30 s: parkingArmed <- true
4  for each limit: confirm over 2 samples to set; clear only below the lower clear
value
5  status <- highest-priority active condition (warning < critical < parked
movement < impact)
6  if status changed:
7      start buzzer pattern; show OLED alert screen; queue alert
8  when buzzer finished: upload alert to Supabase; send Telegram message
9  every 3 s: upload one telemetry row to Supabase
```

3.6 Data Flow

Figure 3.5 shows the context-level data flow. The platform exchanges preferences and ranked results with the user and receives sensor readings and alerts from the ESP32 node.

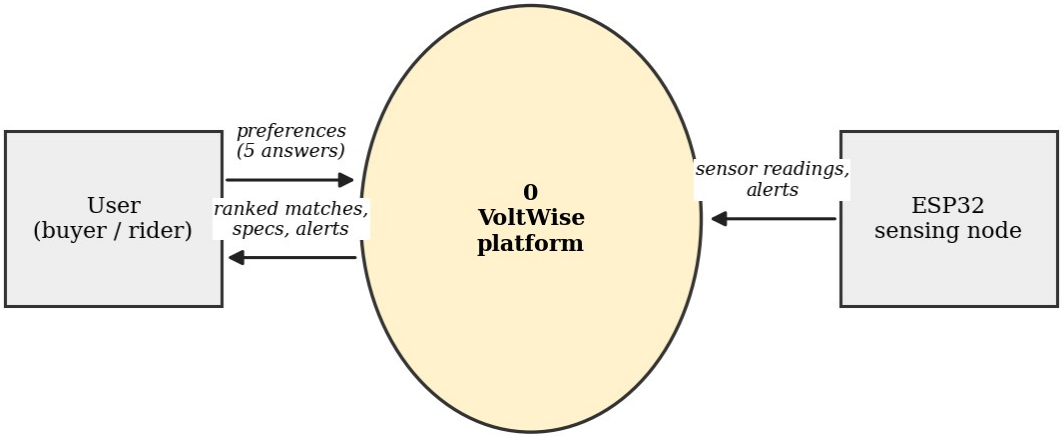


Figure 3.5: Level-0 data flow diagram (context diagram)

Figure 3.6 expands the platform into four processes: collecting rider inputs (1.0), scoring and ranking models using the catalogue data store (2.0), ingesting telemetry into the database (3.0)

and evaluating status and alerts for display (4.0). The status is decided on the device and carried in the telemetry row; the dashboard shows the device status while the node is online.

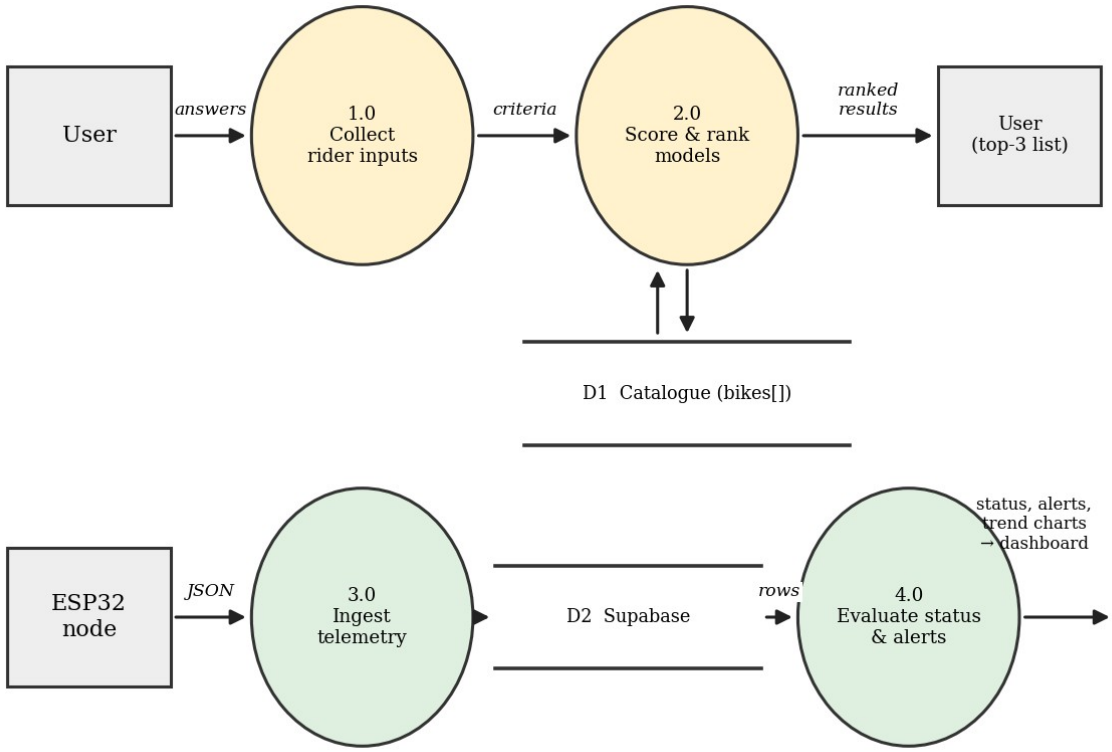


Figure 3.6: Level-1 data flow diagram

3.7 System Modules

The platform is divided into seven modules (see Figure 3.7).

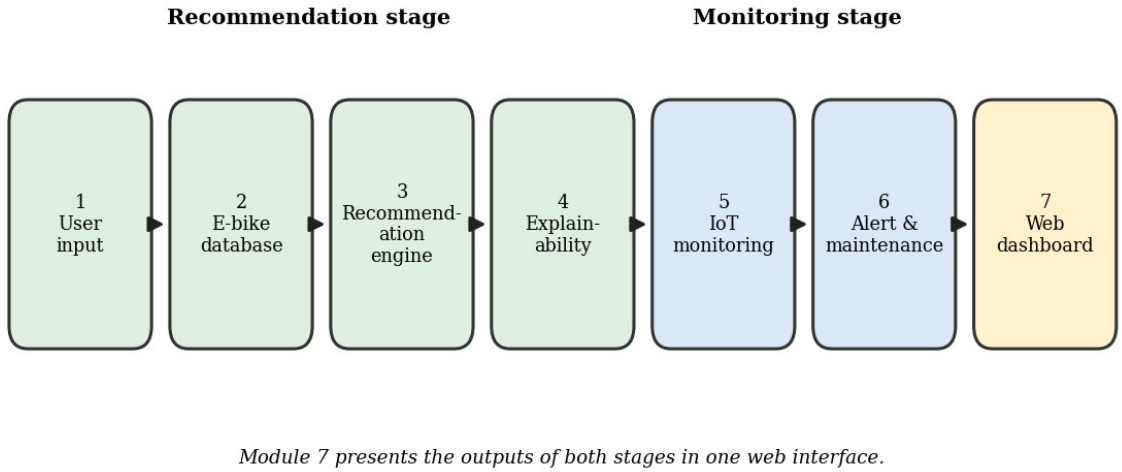


Figure 3.7: Functional modules of the platform

Table 3.2: Module description and status

No.	Module	Function	Status
1	User input	Five-step finder collecting use, budget, range, terrain and priority	Implemented
2	E-bike database	`bikes` array with price, range, battery, motor, speed, weight, terrain, use, priority, rating, features, image	Implemented (8 models, indicative data)
3	Recommendation engine	Five criterion functions, weights, sorting and tie-break	Implemented (rule-based)
4	Explainability	Reason statements and “how the match works” note	Implemented
5	IoT monitoring	ESP32 sensing and Supabase upload; dashboard polling	Implemented for temperature, motion, ambient; voltage/current need the INA219
6	Alert and maintenance	Device alerts, Telegram messages, dashboard alert centre and maintenance insights	Implemented (insights are generic text)
7	Web dashboard	KPI cards, charts, health ring, fleet and analytics views	Implemented; live and simulated modes

3.8 Hardware Connection Design

Figure 3.8 shows the connections taken from the pin assignments declared in the firmware: the I²C devices share GPIO21 (SDA) and GPIO22 (SCL); the DS18B20 bus is on GPIO4 with a 4.7 kΩ pull-up; the buzzer is on GPIO5; the red and green LEDs are on GPIO25 and GPIO26 through 220 Ω resistors. The INA219 is shown dashed because it is not fitted.

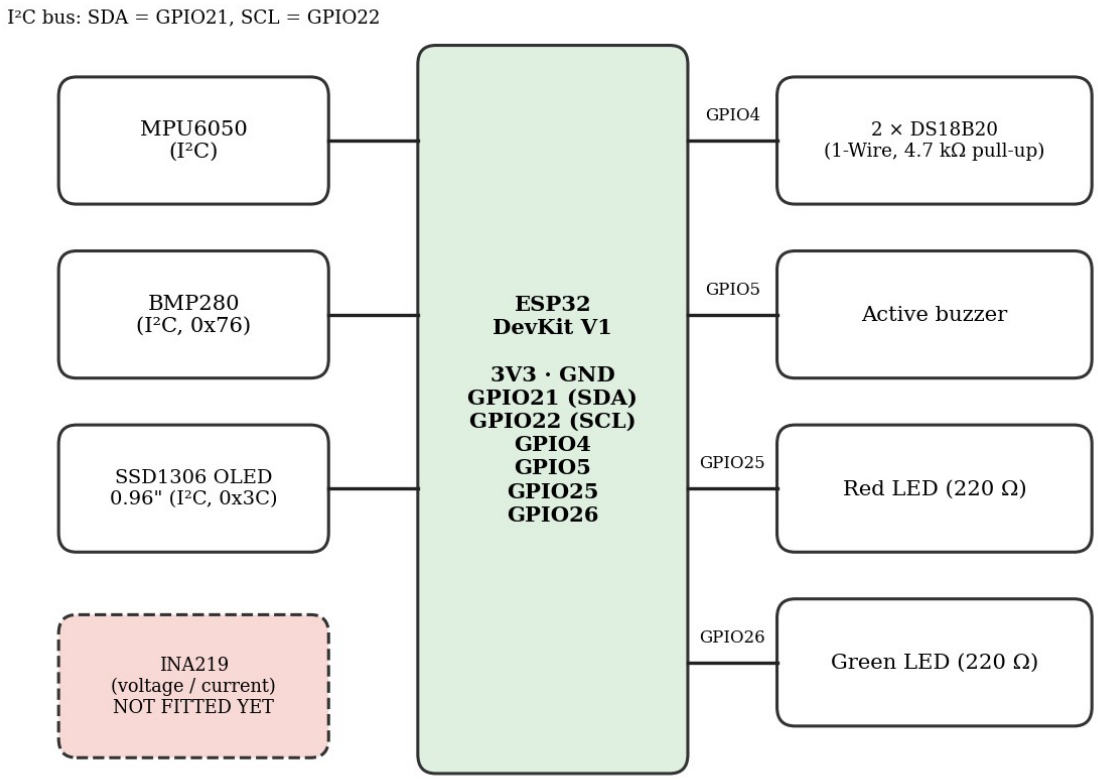


Figure 3.8: Hardware connection diagram of the ESP32 sensing node

3.9 Hardware Requirements

Prototype Hardware (fitted)

Table 3.3: Hardware components of the prototype

Component	Quantity	Role	Interface / pin
ESP32 DevKit V1	1	Controller, Wi-Fi	USB power
DS18B20 temperature sensor	2	Motor and battery temperature	1-Wire, GPIO4, 4.7 kΩ pull-up
MPU6050	1	Vibration, tilt, impact	I²C (GPIO21/22)
BMP280	1	Ambient temperature, pressure	I²C, address 0x76
0.96-inch OLED (SSD1306, 128×64)	1	Local display	I²C, address 0x3C
Active buzzer	1	Audible alarm	GPIO5
Red LED + 220 Ω	1	Alert indication	GPIO25
Green LED + 220 Ω	1	Normal indication	GPIO26
Breadboard, jumper wires	–	Connections	–

Components Listed for Addition

- **INA219** current/voltage monitor – firmware path prepared, not yet wired.

- **NEO-6M GPS module** and **solar panel** – listed by the team as components to add; not part of the present firmware.
- A DS18B20 module/probe set for permanent mounting on the battery and motor.

Development Computer

- Processor: Intel Core i3/i5 class or equivalent
- RAM: 4 GB or more
- Storage: 2 GB free for IDE, libraries and project files
- USB cable (data-capable) for programming the ESP32
- 2.4 GHz Wi-Fi network (the ESP32 does not use 5 GHz)
- Smartphone or PC with a modern browser for the website

3.10 Software Requirements

Table 3.4: Software requirements

Item	Details
Website languages	HTML5, CSS3, JavaScript (ES2020), no external framework
Hosting	Netlify static hosting
Browser	Google Chrome, Mozilla Firefox or equivalent (Web Audio API and canvas required for tone and charts)
Firmware IDE	Arduino IDE with ESP32 board support
Firmware libraries	Wire, OneWire, DallasTemperature, Adafruit_MPU6050, Adafruit_BMP280, Adafruit_SSD1306, WiFi, WiFiClientSecure, HTTPClient, ArduinoJson
Cloud	Supabase project with tables `ebike_telemetry` and `ebike_alerts`
Messaging	Telegram bot created with BotFather
Documentation tools	Microsoft Word for this report

3.11 Database Design

The dashboard reads two tables from Supabase (see Figure 3.9). The column names below are the ones the website source reads; the complete table definitions are in the Supabase project and should be confirmed by the team.

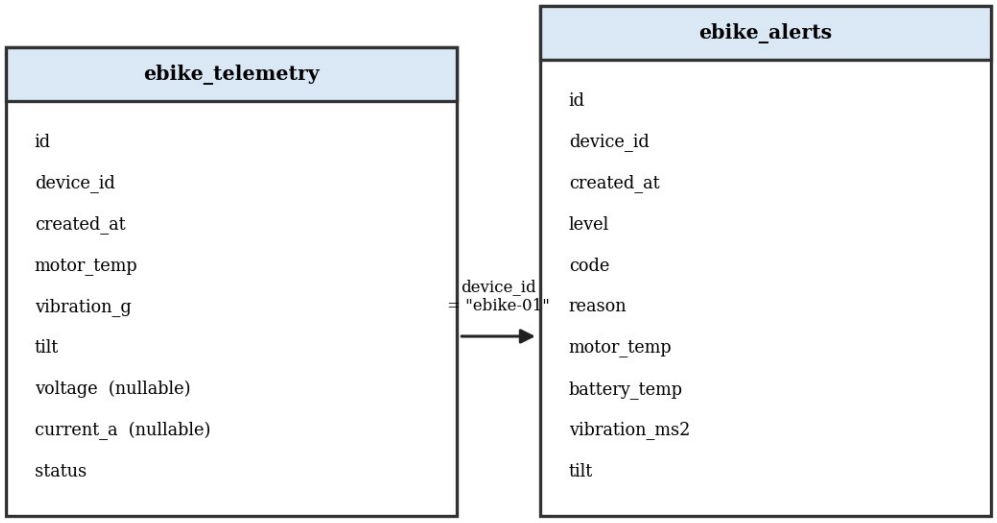


Figure 3.9: Tables used by the dashboard

Table 3.5: Telemetry table columns read by the dashboard

Column	Meaning	Used for
id, created_at	Row identifier and time stamp	Freshness check (older than 15 s = offline)
device_id	Device identifier (`ebike-01`)	Row filter
motor_temp	Motor temperature in °C	Temperature card and chart
vibration_g	Vibration value	Vibration card, movement panel
tilt	Tilt angle in degrees	Frame tilt
voltage, current_a	Battery voltage and current (null while INA219 is not fitted)	Applied only if present
status	NORMAL / WARNING / CRITICAL decided by the device	Overrides the demo status rule

Table 3.6: Alert table columns read by the dashboard

Column	Meaning
level	WARNING or CRITICAL
code, reason	Alert code and readable reason
motor_temp, battery_temp	Temperatures at the time of the alert
vibration_ms2, tilt	Vibration (m/s²) and tilt at the time of the alert
id, created_at	Order and time of the alert

3.12 Component Description

Table 3.7: Roles and key properties of the components

Component	Role in this project	Key property used
ESP32 DevKit V1	Reads sensors, runs alert logic, uploads data	Wi-Fi, I²C, 1-Wire support, GPIO outputs
DS18B20	Motor and battery surface temperature	Digital, unique address, shared bus
MPU6050	Vibration, tilt and impact	3-axis accelerometer (range set to ±8 g in code, filter 21 Hz)
BMP280	Ambient temperature and pressure	I²C address 0x76
SSD1306 OLED 0.96"	Live and alert screens	128×64 pixels, I²C 0x3C
Active buzzer	Audible alert patterns	Driven with `tone()` on GPIO5
Red / green LED	Status indication	220 Ω series resistors
INA219 (planned)	Battery voltage and current	Not fitted; code disabled by `HAS_INA219`
NEO-6M GPS, solar panel (planned)	Location and charging	Not implemented

3.13 User Interface Design

The interface is a single scrolling page with a fixed header and a menu. The design principles are: a clear call to action on the home page (“Find My Ride”), progressive disclosure of the finder (one question per screen), cards for products and results, a dark control-room style for the dashboard, and visible labels such as “Demo data”, “Simulated” and “Offline” wherever the data is not live. On a phone the menu collapses into a button and the cards stack in one column.

3.14 Feasibility Study

Technical Feasibility

The project uses components and tools that are widely available and well documented. The ESP32 DevKit V1, DS18B20, MPU6050, BMP280 and SSD1306 OLED have mature Arduino libraries, and the website needs only a standard browser. Supabase provides a hosted database with a REST interface, and Netlify hosts the static page. The prototype therefore does not require special equipment beyond a computer, a USB cable and a Wi-Fi connection, and it has been assembled and programmed within the time of a mini project.

Economic Feasibility

The sensing node is built from low-cost modules on a breadboard, and the software stack uses free tiers of hosting, database and messaging services during development. No licence fee is needed for the libraries used. Costs would rise if the prototype were moved to a custom board, a waterproof enclosure, a cellular modem or a paid database plan, but these are not required for the demonstration.

Operational Feasibility

A buyer uses the website with five taps and receives a ranked list; no installation is needed. An owner sees a status word (Normal, Warning, Critical), hears a buzzer pattern and receives a Telegram message, which are easy to understand. The dashboard states clearly whether the data is live or simulated, which reduces the risk of misreading it.

Schedule and Risk Considerations

- **Sensor placement and noise:** vibration readings depend on mounting; the firmware calibrates the resting orientation to reduce this problem.
- **Network dependency:** cloud upload needs Wi-Fi; local alarms do not.
- **Credential handling:** keys must be removed from source before public sharing.
- **Data currency:** catalogue values must be refreshed manually.

3.15 Requirement Specification

Functional Requirements

Table 3.8: Functional requirements

ID	Requirement	Module
FR1	The system shall display a catalogue of electric scooter models with price, range, battery, motor, speed and weight.	E-bike database
FR2	The user shall be able to search, filter by category and sort the catalogue.	Catalogue
FR3	The system shall collect five rider inputs and require one choice per step.	User input
FR4	The system shall compute a weighted score for every model and display the top three with a match percentage.	Recommendation engine
FR5	The system shall show reasons for each recommended model.	Explainability
FR6	The user shall be able to compare two or three models and keep a shortlist.	Compare / shortlist
FR7	The node shall read motor and battery temperature, vibration, tilt and ambient conditions every 500 ms.	IoT monitoring
FR8	The node shall classify NORMAL, WARNING or CRITICAL and sound a buzzer, show an OLED alert and set LEDs.	Alert
FR9	The node shall upload telemetry every 3 s and alerts on state change, and send Telegram messages.	IoT monitoring
FR10	The dashboard shall show live data when a recent row exists and otherwise show an offline state.	Web dashboard

Non-Functional Requirements

Table 3.9: Non-functional requirements

Aspect	Requirement
Usability	Simple five-step flow; works on phone and desktop browsers
Transparency	Weights and scoring rules visible; simulated data labelled as such

Aspect	Requirement
Reliability	Local alarms continue without Wi-Fi; hysteresis avoids flickering alerts
Responsiveness	Sensors read every 500 ms; dashboard polls every 2 s; telemetry row every 3 s
Privacy	Finder answers and shortlist stay in the browser (localStorage)
Security (needed)	Credentials must be kept out of public source; database access should be restricted by row-level security
Maintainability	Single-file website and one firmware sketch with configuration constants at the top

3.16 Summary

This chapter presented the existing and proposed systems, the architecture, the workflow and UML diagrams, the data flow, the system modules, the hardware connection and requirements, the database, the components, the user interface, the feasibility study and the requirement specification. The next chapter explains how the design was implemented and how the system works.

CHAPTER 4
IMPLEMENTATION AND METHODOLOGY

4.1 Methodology

The project followed an incremental, modular approach in which the website and the hardware node were built in parallel and joined through a cloud database.

1. **Requirement analysis.** Buyer needs (simple inputs, comparison, explanation) and owner needs (temperature, vibration, impact and movement alerts) were listed.
2. **Catalogue preparation.** Eight electric scooter models from different brands were entered with indicative specifications.
3. **Scoring design.** Five criteria and fixed weights were chosen so that the ranking can be explained in one sentence.
4. **Web development.** The catalogue, finder, comparison, shortlist and dashboard were written as a single HTML/CSS/JavaScript page.
5. **Hardware assembly.** The ESP32 node was assembled on a breadboard with the sensors listed in Chapter 3.
6. **Firmware development.** Sensor reading, orientation calibration, alert logic, buzzer patterns, OLED screens and network uploads were programmed.
7. **Integration.** A small script in the website polls Supabase and passes new rows to the dashboard's `updateTelemetry()` function.
8. **Testing and evidence.** Website functions and alert behaviour are tested and captured (Section 5.6).
9. **Deployment.** The website was published on Netlify and viewed from a phone.

4.2 Implementation

4.2.1 Website Structure and Technology

The website is implemented as a single HTML file that contains its styles and two script blocks. The first script holds the product data, the recommendation engine and the dashboard logic; the second script is a "bridge" that reads live data from Supabase. No front-end framework is used. Product photographs are embedded in the file, so the page does not depend on image servers. The navigation bar links to the sections Home, Discover, Recommend, Compare, Monitor, Fleet, Analytics, Alerts, Architecture and Hardware, followed by roadmap and about sections. Figure 4.1 shows the home section, and Figure 4.2 shows the catalogue as captured from the deployed site on a phone.

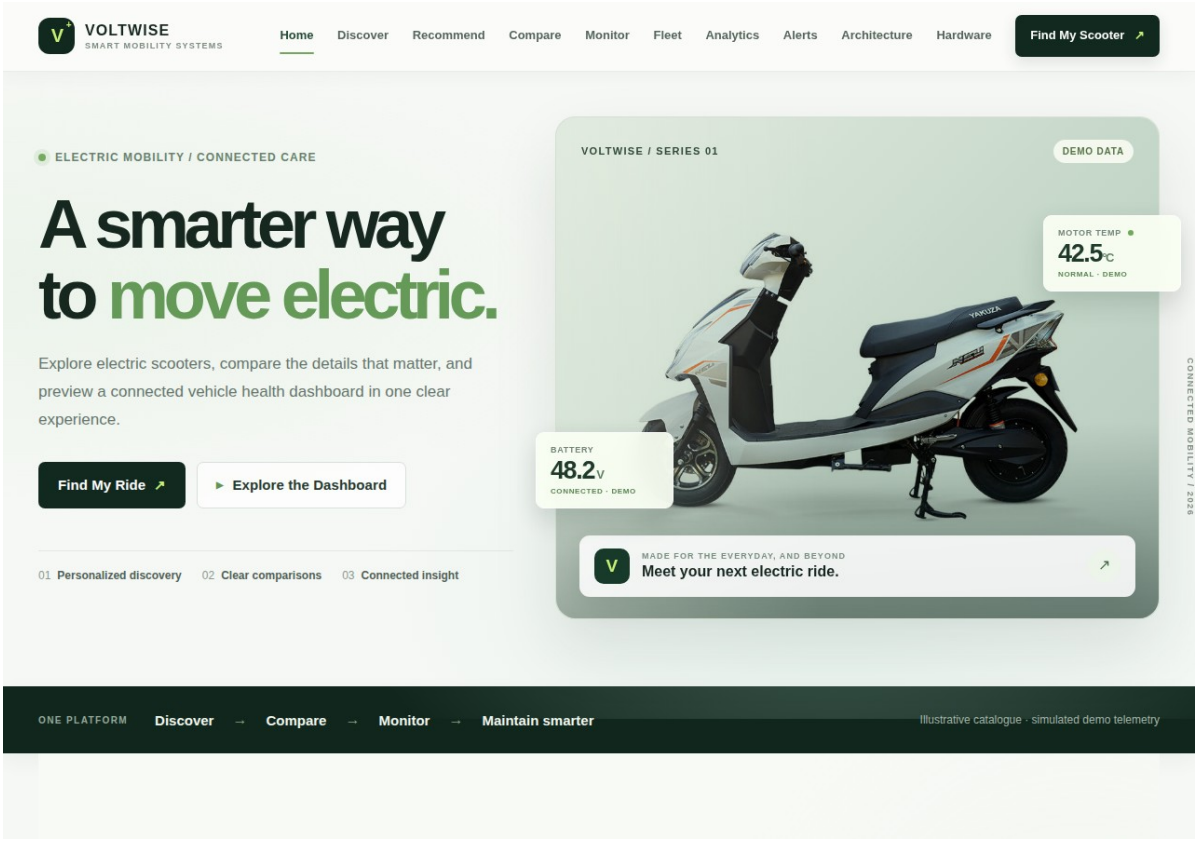


Figure 4.1: VoltWise home page (desktop view rendered from the supplied source file)

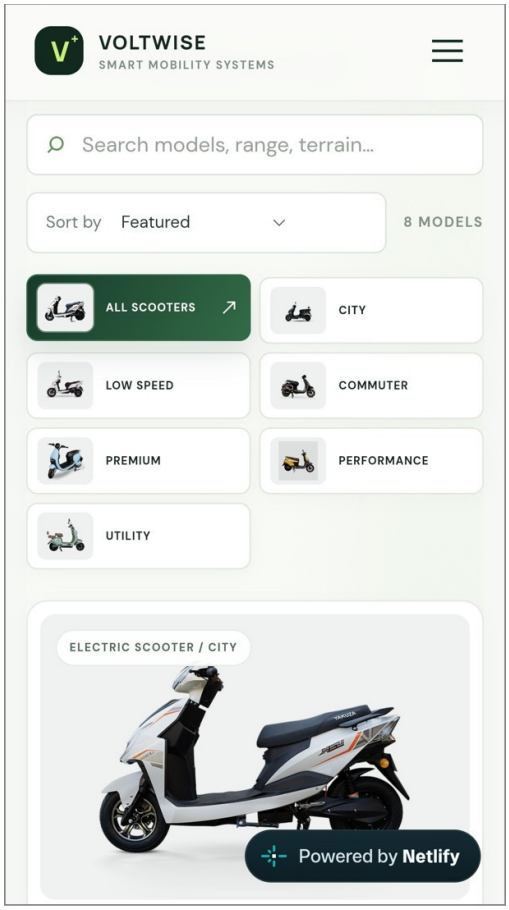


Figure 4.2: Catalogue with search, sort and category filters on a phone (screenshot of the deployed site)

4.2.2 E-bike Database Module

The catalogue is a JavaScript array named `bikes`. Each record stores `id`, `brand`, `name`, `category`, `price` (rupees), `range` (km), `battery`, `motor`, `topSpeed`, `weight` (kg), `terrain` (list), `use` (list), `priority` (list), `rating`, a short description and an image. The values are described in the page itself as indicative and for illustration; the buyer is asked to confirm current details with each manufacturer.

Table 4.1: Catalogue records implemented in the website

Model	Category	Price	Range (km)	Battery	Motor	Top speed	Weight (kg)
Yakuza Neu	City	₹39,760	45	48 V	250 W BLDC	25 km/h	68
Komaki X-One	City	₹37,499	70	$LiFePO_4$	Hub motor	25 km/h	72
AMO Mobility Feisty EV	Low speed	₹54,999	75	60 V	249 W BLDC	< 25 km/h	58
Gigaglitz Coco	Premium	₹99,900	60	60 V	BLDC	25 km/h	64
Yakuza Viraj	Commuter	₹44,800	45	48 V	250 W BLDC	25 km/h	76

Model	Category	Price	Range (km)	Battery	Motor	Top speed	Weight (kg)
BGauss C12i Max 2.0	Performance	₹1,24,999	123	2.7 kWh	2.5 kW hub	60 km/h	106
NUV S2 Nyra	Utility	₹1,19,999	65	Li-ion	BLDC	25 km/h	84
OKAI Ceetle Pro	Premium	₹1,39,999	68	Li-ion	BLDC	25 km/h	78

4.2.3 User Input Module

The finder shows one question at a time with a progress bar (Step 1 of 5 to Step 5 of 5). A visitor must choose one option before continuing; otherwise the message “Choose one option to continue.” appears. Figure 4.3 shows the third step. The five questions and options are given in the table below. Note that the finder collects five inputs; it does not collect rider height or weight.

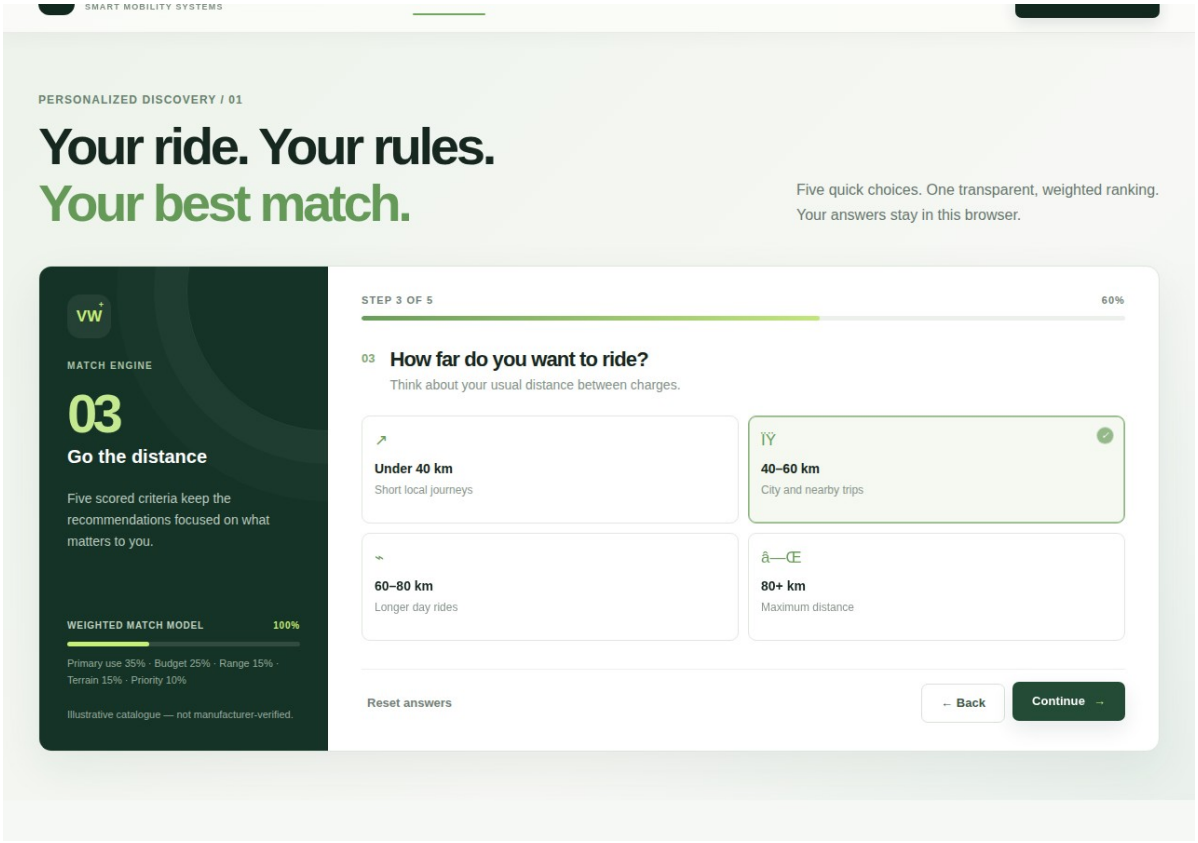


Figure 4.3: Recommendation finder, step 3 of 5 (desktop view rendered from the supplied source file)

Table 4.2: Finder questions and options

No.	Criterion (key)	Question	Options
1	Use (`use`)	What will you use your e-scooter for?	City / Commuting; Campus / Short Trips; Long Distance; Delivery / Utility; Mixed Use
2	Budget (`budget`)	What is your budget?	Under ₹60,000; ₹60,000–₹90,000; ₹90,000–₹1,20,000; Above ₹1,20,000
3	Range (`range`)	How far do you want to ride?	Under 40 km; 40–60 km; 60–80 km; 80+ km

No.	Criterion (key)	Question	Options
4	Terrain (`terrain`)	What roads will you ride?	City roads; Mixed; Uneven
5	Priority (`priority`)	What matters most while riding?	Comfort; Range; Performance; Utility; Balanced

4.2.4 Recommendation Engine

The engine is **rule-based multi-criteria scoring**. It is not a trained machine-learning model, and the page itself labels the results “Rule-based score · not a trained model”. For every model, five criterion scores between 0 and 100 are computed and combined with fixed weights (use 35%, budget 25%, range 15%, terrain 15%, priority 10%, total 100%). The workflow is shown in Figure 4.4.

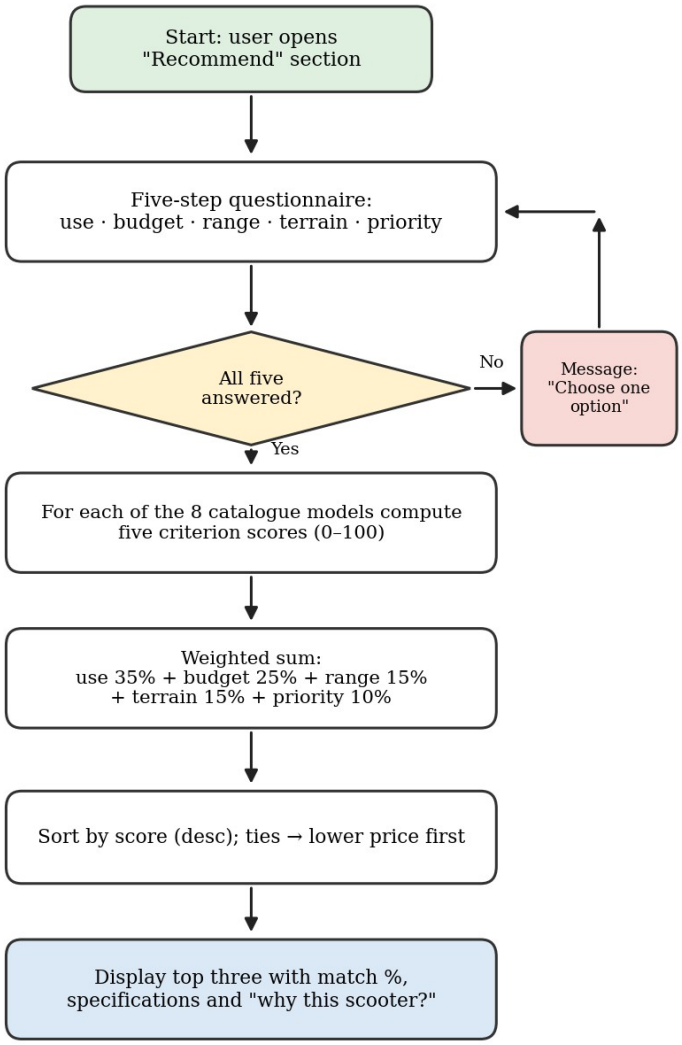


Figure 4.4: Workflow of the recommendation engine

The final score of a model is

$$Score = 0.35 \cdot S_{use} + 0.25 \cdot S_{budget} + 0.15 \cdot S_{range} + 0.15 \cdot S_{terrain} + 0.10 \cdot S_{priority}$$

where each S is the criterion score. The result is rounded to one decimal for ranking and to a whole number for the displayed "% match". Models are sorted by score in descending order; when two models have the same score, the cheaper one is listed first. The five criterion functions are described below.

Table 4.3: Criterion scoring rules implemented in the website

Criterion	Rule (as coded)	Possible scores
Use (35%)	100 if the selected use appears in the model's `use` list; otherwise 90 if the answer is "City / Commuting" and the category is City or Commuter; otherwise 70 if the answer is "Mixed Use"; otherwise 35	100, 90, 70, 35
Budget (25%)	Model price is placed in a band (below ₹60,000; ₹60,000–₹89,999; ₹90,000–₹1,20,000; above ₹1,20,000). The score depends on the distance between the model's band and the chosen band	0 bands away: 100; 1: 74; 2: 48; 3: 24
Range (15%)	Model range is placed in a band (below 40 km; 40–59; 60–80; above 80). The score depends on the band distance to the chosen band	100, 76, 50, 25
Terrain (15%)	100 if the selected terrain is in the model's `terrain` list; otherwise 72 if the model supports "Mixed"; otherwise 38	100, 72, 38
Priority (10%)	100 if the selected priority is in the model's `priority` list; otherwise 78 for "Balanced"; for "Range", the model's range ÷ 90 × 100 capped at 100; otherwise 48	100, 78, range-based, 48

Worked example. The finder was run on the supplied source with the answers "City / Commuting", "Under ₹60,000", "40–60 km", "City roads" and "Comfort". The criterion scores and the final scores produced by the code are listed below. The two highest models tie at 100.0, and the cheaper Yakuza Neu is placed first by the tie-break rule.

Table 4.4: Scores produced by the engine for the worked example

Rank	Model	Use	Budget	Range	Terrain	Priority	Score
1	Yakuza Neu	100	100	100	100	100	100.0
2	Yakuza Viraj	100	100	100	100	100	100.0
3	Komaki X-One	100	100	76	100	100	96.4
4	AMO Mobility Feisty EV	100	100	76	100	100	96.4
5	Gigaglitz Coco	100	48	76	100	100	83.4
6	OKAI Ceetle Pro	100	24	76	100	48	72.2
7	BGauss C12i Max 2.0	100	24	50	72	48	64.1
8	NUV S2 Nyra	35	48	76	100	48	55.4

The engine has the following properties and limitations that should be understood when reading its output:

- The weights are fixed and are not chosen by the user.
- Only price, range, terrain, use and priority tags influence the score; top speed, weight, battery type and the displayed rating do not.
- The score is a measure of how well a model matches the stated answers on a 0–100 scale. It is **not a probability** and not a prediction of satisfaction.
- Several models can reach the same score, which is why the tie-break rule exists.
- The catalogue is small and indicative; the ranking is only as good as these data.

4.2.5 Explainability Module

Below each of the top three results the page shows a “Why this scooter?” list. The sentences are generated from the criterion scores: a criterion with a high score produces a positive statement (for example, a strong fit for the chosen use, or a fit with the selected budget), and a lower score produces a more cautious statement. A note under the results states the weights and that each contribution is visible. This gives the buyer a plain-language reason in addition to the percentage.

4.2.6 Catalogue, Comparison and Shortlist

The catalogue supports a text search (over name, brand, category, description, battery, motor, use and terrain), seven category chips and sorting. The comparison section allows two or three models to be added and then displays them side by side with price, range, battery, motor, top speed, weight, terrain, use case and highlights (Figure 4.5). Models can be saved to a shortlist (“garage”) and a detail view opens a model’s highlights. The shortlist, the selected scooter for monitoring and the buzzer preference are stored in the browser’s `localStorage`, so no account or server storage is used.

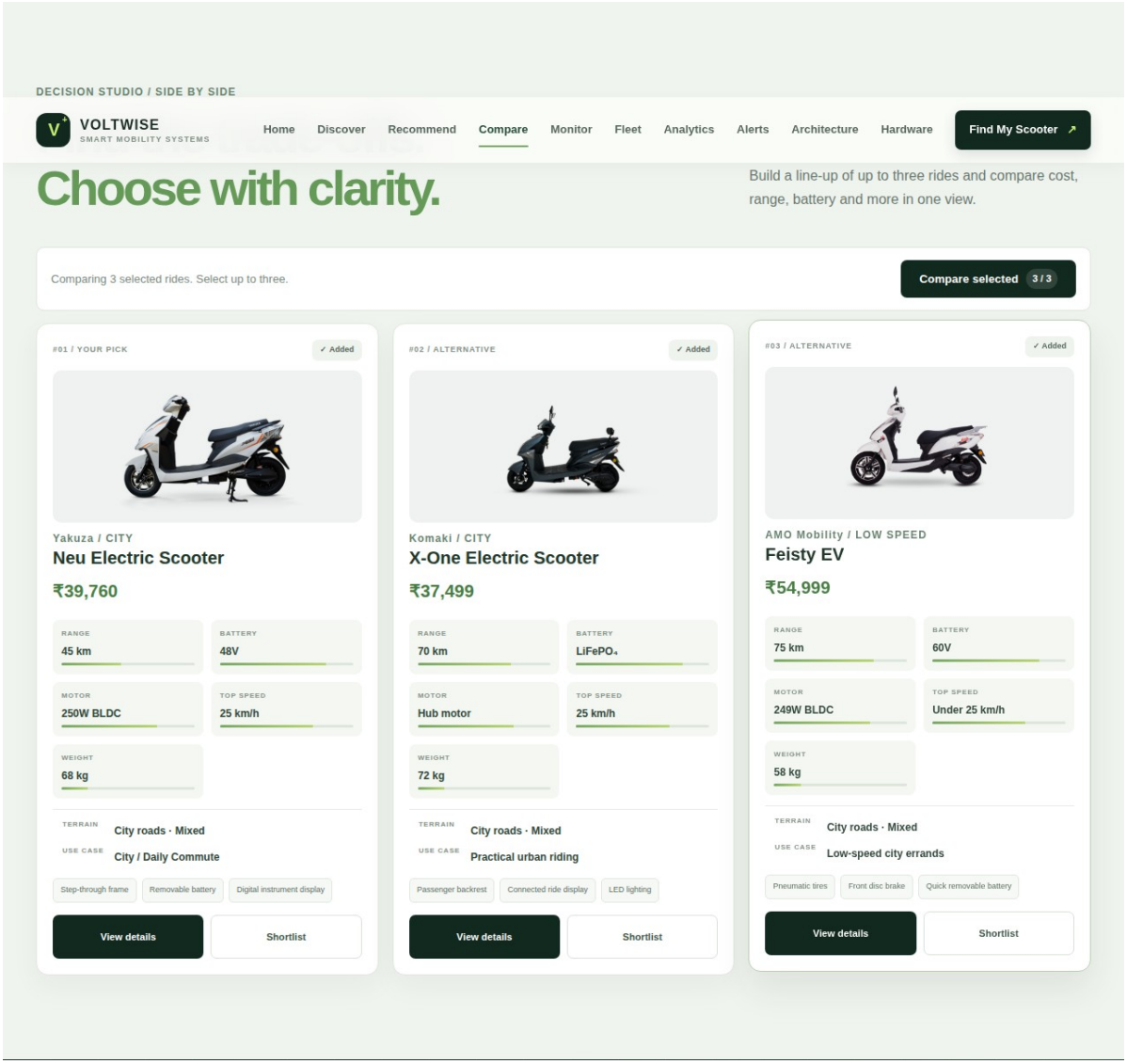


Figure 4.5: Side-by-side comparison of three models (desktop view rendered from the supplied source file)

4.2.7 ESP32 Sensing Node

The prototype (Figure 4.6) is assembled on a breadboard. The ESP32 DevKit V1 is connected to the sensors and outputs as in Figure 3.8. The firmware identifies the device as `ebike-01` and prints a start-up report on the serial port at 115200 baud.

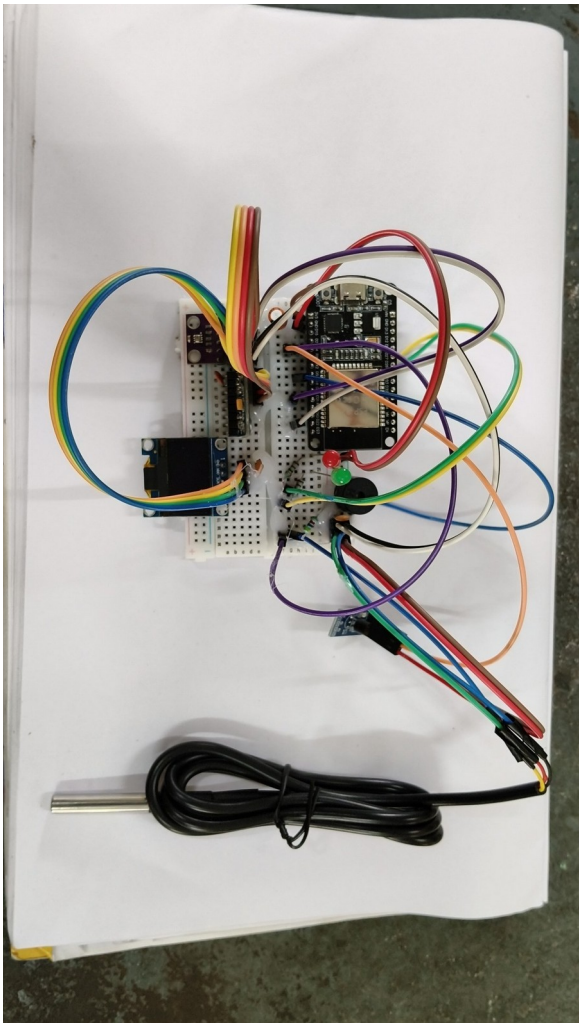


Figure 4.6: Hardware prototype: ESP32 on a breadboard with MPU6050, BMP280, OLED, buzzer, red and green LEDs and a DS18B20 probe

Pin and address assignment

Table 4.5: Pin and address assignment in the firmware

Function	Pin / address	Notes
DS18B20 (2 sensors)	GPIO4	Single 1-Wire bus; sensor index 0 = motor, index 1 = battery
I²C bus	GPIO21 (SDA), GPIO22 (SCL)	MPU6050, BMP280, OLED
BMP280	0x76	Some boards use 0x77
OLED SSD1306	0x3C	128 × 64 pixels
Buzzer	GPIO5	Driven with `tone()`
Red LED	GPIO25	Through 220 Ω
Green LED	GPIO26	Through 220 Ω

Sensor processing

- **Temperatures.** `requestTemperatures()` is called every 500 ms and the motor and battery probe temperatures are read in °C at 10-bit resolution. The BMP280 supplies ambient temperature and pressure (in hPa).
- **Orientation calibration.** At start-up the firmware averages 100 accelerometer samples (8 ms apart) to learn the direction of gravity and the resting acceleration magnitude. Because of this, the MPU6050 may be mounted flat or vertically; the code states that the board was mounted vertically.
- **Vibration.** The vibration value is the length of the difference between the current acceleration vector and the resting vector. This also detects sideways or rotational movement that leaves the total magnitude unchanged.
- **Tilt.** The tilt angle is the angle between the current gravity vector and the calibration vector, so it is independent of how the board is mounted.
- **Impact.** An impact is flagged when the vibration value exceeds 12.0; the flag stays visible for 8 s.
- **Parking mode.** If the vehicle has been still (vibration below 0.8) for 30 s, parking mode is armed. Afterwards, vibration above the warning limit or tilt above 30° for two consecutive samples is reported as movement while parked.
- **Voltage and current.** Compiled only when `HAS_INA219` is true; at present the flag is false, so the values are not-a-number and are not sent as measurements.

Alert thresholds

Table 4.6: Thresholds used by the firmware

Quantity	Warning (clear)	Critical (clear)	Other
Motor temperature	> 55 °C (< 52 °C)	> 70 °C (< 66 °C)	–
Battery temperature	> 45 °C (< 42 °C)	> 55 °C (< 51 °C)	–
Vibration	> 2.5 (< 1.8)	> 6.0 (< 4.5)	Vibration value in m/s²
Impact	–	> 12.0	Latched 8 s
Tilt while parked	–	> 30°	Parking armed after 30 s still
Voltage (if INA219 fitted)	< 11.0 V (> 11.6 V)	< 10.0 V (> 10.5 V)	Not active
Current (if INA219 fitted)	> 2000 mA	–	Not active

Alert logic

A condition must hold for two consecutive 500 ms samples (`CONFIRM_SAMPLES`) before an alert flag is set, and it clears only when the reading falls below a lower “clear” value. This hysteresis prevents the status from flickering when a reading sits on a limit. The resulting status is the highest-priority active condition: warnings first, then critical temperature or vibration

conditions, then movement while parked (code `THEFT`) and finally impact (code `IMPACT`). The status, a short reason and a code are stored, and actions are taken only when the status changes.

Table 4.7: Alert codes and local responses

Code	Level	Reason text	Buzzer pattern
MOTOR_TEMP_WARN	Warning	Motor temperature elevated	1200 Hz, 120 ms on / 400 ms off, 2 beeps
BATT_TEMP_WARN	Warning	Battery temperature elevated	as above
VIB_WARN	Warning	Abnormal vibration	as above
MOTOR_TEMP_CRIT	Critical	Motor overheating	2000 Hz, 400/200 ms, 5 beeps
BATT_TEMP_CRIT	Critical	Battery overheating	as above
VIB_CRIT	Critical	Severe vibration	as above
THEFT	Critical	Movement while parked	2200 Hz, 250/120 ms, 6 beeps
IMPACT	Critical	Impact detected	2600 Hz, 90/60 ms, 8 beeps

The buzzer is driven by a non-blocking pattern player so that the main loop continues to read sensors while the tone is playing. Network sending can block for several seconds, so an alert is first queued and is uploaded and sent to Telegram only after the buzzer pattern has finished. The OLED shows a live screen and, for 2.5 s (warning) or 4 s (critical), an alert screen; the LEDs show the status colour. The firmware flow is summarised in Figure 4.7.

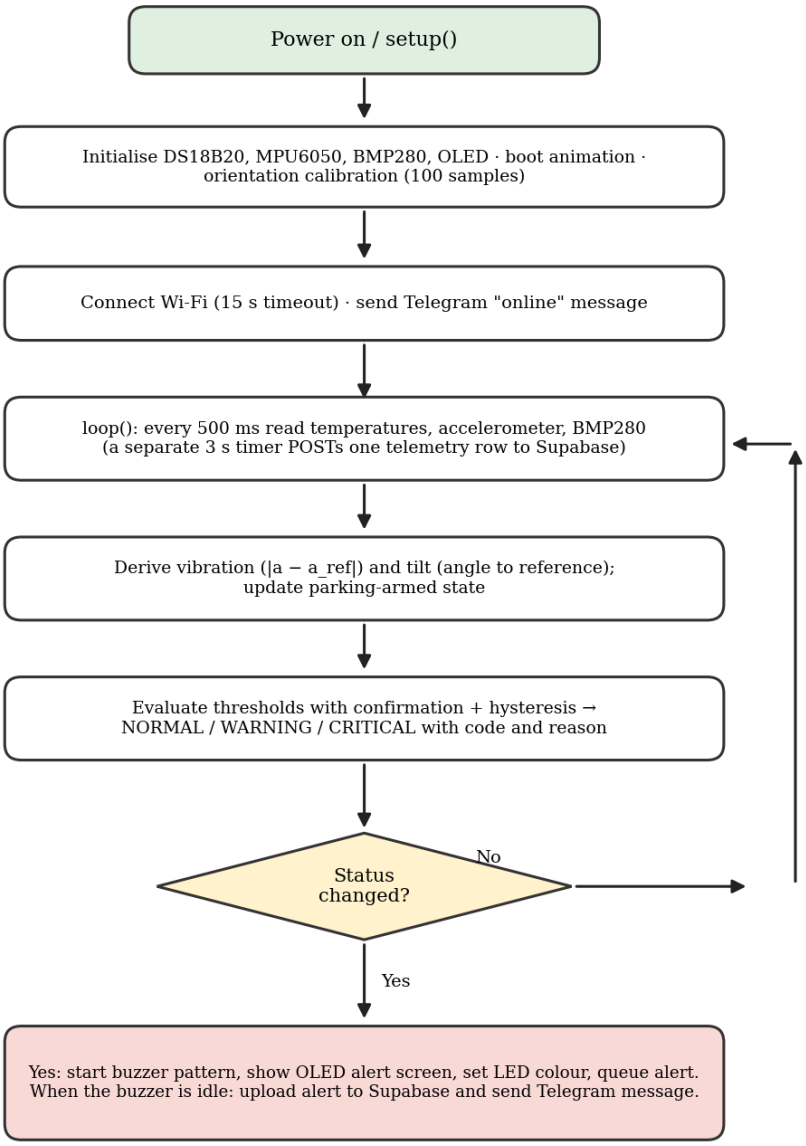


Figure 4.7: Flow of the ESP32 firmware

4.2.8 Communication and Backend

The ESP32 connects to a 2.4 GHz Wi-Fi network in station mode with a 15-second timeout; if the connection fails the node continues to run offline, and the display and buzzer still work. When connected, it sends a Telegram message announcing that the monitor is online. Telemetry is posted to the Supabase table `ebike_telemetry` every 3 s, and alerts are posted to `ebike_alerts` on every change of state. A Telegram message is sent for each warning and critical alert, and a “cleared” message when the state returns to normal. The firmware also defines a 60 s cooldown for repeated Telegram messages with the same code.

On the website side, the bridge script requests the newest telemetry row for device `ebike-01` from the Supabase REST interface every 2 s. If the row is older than 15 s the page treats the

device as offline. The columns read by the page are `motor_temp`, `vibration_g`, `tilt`, `voltage` and `current_a`, plus `status` and `created_at`; alert rows are read with `level`, `code`, `reason`, `motor_temp`, `battery_temp`, `vibration_ms2` and `tilt`. Voltage and current are applied only when they are present, so the dashboard keeps its previous value instead of showing an invented number. New alerts are added to the alert centre; the first fetch after a page load only records the latest alert id so that old alerts are not replayed. Because the browser reads from Supabase, the browser and the ESP32 need not be on the same network.

4.2.9 Alert and Maintenance Module

The alert centre lists alerts with their level, title, message and time, can filter them by level and can be cleared. A short tone for warning and critical alerts can be switched on or off in the browser (Web Audio API). The maintenance insights panel shows short suggestions that depend on the current status (for example, to inspect the vehicle after a critical event). These suggestions are generic rules, not diagnostics, and the page says so.

4.3 Working Process of the System

This section explains, step by step, how the system works from the moment a visitor opens the website to the moment an alert reaches the owner. It is written from the supplied source code and the evidence available.

4.3.1 Working Process of the Recommendation Module

1. **Open the site.** The browser loads the single HTML file. The script builds the catalogue cards, the category chips, the first question of the finder, the shortlist area, the fleet and the dashboard (function `setupRuntime`).
2. **Browse the catalogue.** The visitor may type in the search box, tap a category chip or change the sort order. Each action re-renders the list from the `bikes` array.
3. **Start the finder.** The “Recommend” section shows “Step 1 of 5”. The answers are stored in `state.answers` when an option is chosen.
4. **Validate and continue.** Pressing Continue without a choice shows “Choose one option to continue.” After the fifth answer the engine runs.
5. **Score all models.** For each of the eight models the five criterion functions are evaluated and multiplied by their weights.
6. **Rank.** Models are sorted by score, with the lower price first when scores are equal.
7. **Show results.** The top three are displayed with match percentage, specifications and reasons, and the page scrolls to the results.
8. **Act on a result.** The visitor can shortlist, view details, compare, or choose the model for monitoring. The chosen model’s name and picture then appear in the monitoring section.

The worked example in Section 4.2.4 follows exactly these steps; the next subsection gives three further scenarios computed by the same code.

4.3.2 Further Recommendation Scenarios

The following scenarios were obtained by running the supplied source with different answers. They demonstrate how the ranking changes with the inputs. Only the first four models of each ranking are shown.

Scenario A – long-distance rider with a mid-range budget

Answers: Long Distance; ₹90,000–₹1,20,000; 80+ km; Mixed; Range.

Table 4.8: Scenario A ranking

Rank	Model	Use	Budget	Range	Terrain	Priority	Score
1	BGauss C12i Max 2.0	100	74	100	100	100	93.5
2	OKAI Ceetle Pro	100	74	76	100	76	87.5
3	Komaki X-One	100	48	76	100	100	83.4
4	Yakuza Viraj	100	48	50	100	100	79.5

The BGauss C12i Max 2.0 ranks first. Its claimed range of 123 km falls in the same band as the requested 80+ km, and its terrain and priority tags match, which outweighs the fact that its price (₹1,24,999) lies in the band above the chosen budget.

Scenario B – utility use with a high budget

Answers: Delivery / Utility; Above ₹1,20,000; 60–80 km; City roads; Utility.

Table 4.9: Scenario B ranking

Rank	Model	Use	Budget	Range	Terrain	Priority	Score
1	NUV S2 Nyra	100	74	100	100	100	93.5
2	OKAI Ceetle Pro	35	100	100	100	100	77.3
3	Gigaglitz Coco	35	74	100	100	48	65.5
4	BGauss C12i Max 2.0	35	100	76	72	48	64.3

The NUV S2 Nyra ranks first because it is the only model tagged for delivery or utility use and also matches the budget, range, terrain and priority.

Scenario C – short campus trips with a low budget

Answers: Campus / Short Trips; Under ₹60,000; Under 40 km; City roads; Balanced.

Table 4.10: Scenario C ranking

Rank	Model	Use	Budget	Range	Terrain	Priority	Score
1	Gigaglitz Coco	100	48	50	100	100	79.5
2	Yakuza Neu	35	100	76	100	100	73.7

Rank	Model	Use	Budget	Range	Terrain	Priority	Score
3	Yakuza Viraj	35	100	76	100	100	73.7
4	Komaki X-One	35	100	50	100	100	69.8

Only the Gigaglitz Coco carries the campus tag, so it ranks first even though its price is above the chosen budget band; this shows how the large weight on use (35%) can outweigh budget (25%). The two Yakuza models tie at 73.7 and are ordered by price. This behaviour is a property of the fixed weights and is stated as a limitation.

4.3.3 Working Process of the Monitoring Node

1. **Power-up.** The ESP32 starts, prints a banner on the serial port and initialises the DS18B20 bus, MPU6050, BMP280 and OLED; missing devices are reported as “NOT FOUND”.
2. **Boot animation and calibration.** The OLED shows a start-up animation. The firmware then averages 100 accelerometer samples to learn the resting gravity vector. The board should therefore be still and in its final mounting during this step.
3. **Wi-Fi connection.** The node tries to join the configured network for up to 15 s. If it fails it continues offline.
4. **Online message.** If connected, it sends a Telegram message stating the device name, IP address and what is monitored.
5. **Main loop.** Every 500 ms the node reads sensors, updates the parking state, evaluates thresholds and updates the OLED, LEDs and buzzer. The serial port prints a status line about once per second.
6. **Alert handling.** When the status changes to WARNING or CRITICAL the buzzer pattern starts, the OLED alert screen appears and the alert is queued. When the buzzer has finished, the alert is uploaded to Supabase and a Telegram message is sent.
7. **Telemetry upload.** Every 3 s one telemetry row is posted to Supabase.
8. **Recovery.** When readings fall below the clear values for two samples, the status returns to NORMAL and a “cleared” message is sent.

4.3.4 Working Process of the Dashboard

1. **Start.** The page shows the demo simulator in normal mode until the bridge confirms live data.
2. **Polling.** Every 2 s the bridge requests the newest row for `ebike-01`.
3. **Freshness check.** If the row is younger than 15 s the page switches to live mode: labels become “ESP32 online / live hardware”, the simulator stops updating and the device status replaces the demo status rule.
4. **Display.** Temperature, vibration and tilt cards, the chart and the movement panel are updated through `updateTelemetry()`.

5. **Alerts.** New alert rows are added to the alert centre; older alerts are not replayed after a reload.
6. **Offline.** If no recent row exists, or two requests fail, the page returns to the offline state.

4.3.5 End-to-End Working Process

The complete flow is: the rider chooses a model with the finder; the same model is shown on the monitoring section; the sensing node on the vehicle measures and judges its own condition; the result is stored in the cloud; the dashboard and Telegram inform the owner. The recommendation and monitoring modules are independent in code, but they share one interface and one selected-vehicle context.

4.3.6 Event-to-Response Walk-through

The table below traces what the implementation does for typical events. It is derived from the firmware logic and the dashboard source; each row is a behaviour to be demonstrated and captured during testing.

Table 4.11: Event-to-response mapping

Event	Detection rule	Local response	Remote response
Motor above 55 °C	Two samples > 55 °C	WARNING: 1200 Hz two beeps, OLED alert 2.5 s, LED	Alert row, Telegram message, dashboard alert
Motor above 70 °C	Two samples > 70 °C	CRITICAL: 2000 Hz five beeps, OLED alert 4 s	Alert row, Telegram, dashboard
Battery above 45 °C / 55 °C	Two samples above limit	WARNING / CRITICAL as above	As above
Shaking	Vibration > 2.5 (warning), > 6.0 (critical)	Buzzer by level	As above
Impact	Vibration > 12.0	CRITICAL: 2600 Hz eight fast beeps; impact shown for 8 s	As above
Moved or tilted while parked	Armed after 30 s still; vibration > 2.5 or tilt > 30° for two samples	CRITICAL: 2200 Hz six beeps	As above
Wi-Fi unavailable	No connection within 15 s	Node keeps reading and alarming locally	No upload until connected
No telemetry for 15 s	Row older than 15 s	–	Dashboard shows Offline

4.4 Summary

This chapter described the methodology, the implementation of each module of the website, the ESP32 sensing node, the communication and backend, and the working process of the recommendation module, the monitoring node and the dashboard, with further recommendation scenarios and an event-to-response mapping. The results obtained are discussed in the next chapter.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Experimental Setup

The results in this chapter come from three sources: (i) the supplied website file, which was executed in a desktop browser (headless Chromium) to obtain the recommendation outputs and the demonstration screens; (ii) the phone screenshots of the deployed website on Netlify, captured while the ESP32 node was not connected; and (iii) the photograph and firmware of the hardware prototype. No sensor readings, accuracy percentages or performance figures have been invented. Live sensor behaviour is demonstrated on the prototype, and the report states clearly where a feature is simulated.

5.2 Recommendation Results

For the sample answers “City / Commuting”, “Under ₹60,000”, “40–60 km”, “City roads” and “Comfort”, the engine ranked the Yakuza Neu Electric Scooter first, the Yakuza Viraj Electric Scooter second and the Komaki X-One Electric Scooter third, as shown in Figure 5.1. The first two models both obtain a score of 100.0; the cheaper Neu is placed first by the tie-break rule. The detailed criterion scores are given in Chapter 4, and three further scenarios with different answers are given in the working-process section of that chapter.

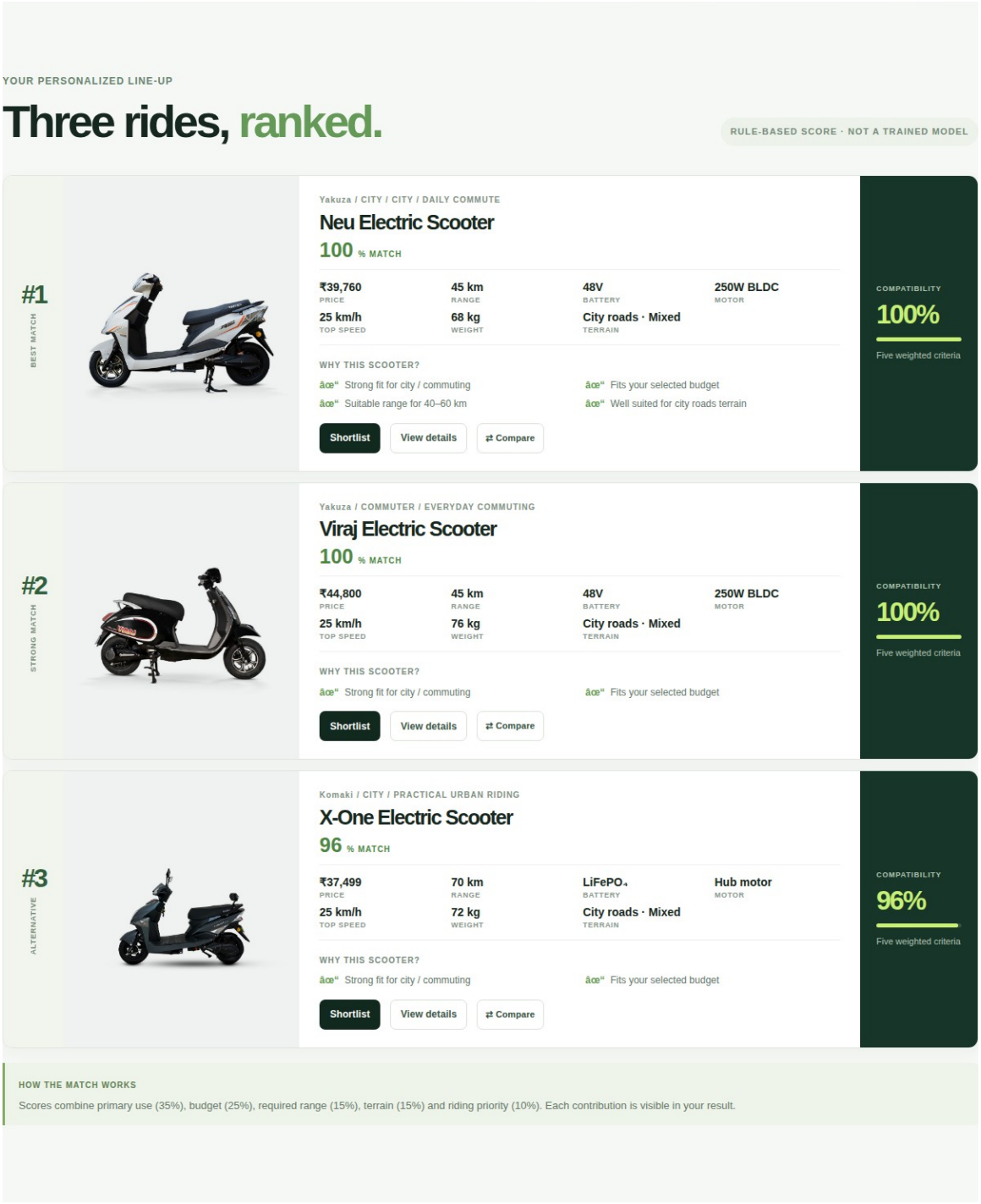


Figure 5.1: Ranked recommendation results for the sample answers

The results show that the ranking responds in a sensible and explainable way to the inputs: a rider who wants city commuting on a low budget receives low-cost city models, a long-distance rider with a mid-range budget receives the long-range BGauss C12i Max 2.0, and a rider who needs a utility vehicle receives the NUV S2 Nyra. The ranking also shows a property of the fixed weights: because use carries 35%, a model tagged for the exact use can outrank a cheaper model that misses the use tag, even when the budget band differs.

5.3 Dashboard Results

When the ESP32 node was not connected, the dashboard displayed “Offline”, “No live data”, “No physical node connected to this page” and dashes in place of values, and the trend chart reported that there were no samples in the selected window (see Figure 6.1). This confirms the designed behaviour that the page does not display invented values without a recent row. It should not be read as a hardware fault. The simulated fleet of twelve units is shown separately in Figure 6.2 and is labelled as demonstration data.

5.4 Monitoring Evidence

The hardware prototype is shown in Figure 4.6. The sensing behaviour described in Chapter 4 (temperature, vibration, tilt and impact detection, buzzer patterns, OLED alert screens and the Supabase and Telegram alerts) is implemented in the firmware and is demonstrated on the prototype itself. This report does not include recorded sensor readings or accuracy figures; every value shown on the dashboard is either marked as simulated or comes from the live bridge described in Chapter 6.

5.5 Implementation Status Summary

Table 5.1: Status of features at the time of writing

Feature	Status	Evidence available
Catalogue, search, filter, sort	Implemented, executed	Screenshot (phone); source
Five-step finder and ranking	Implemented, executed	Run on supplied source; results figure
Comparison, shortlist	Implemented, executed	Run on supplied source
Dashboard layout and offline state	Implemented, evidenced	Phone screenshots
Dashboard demonstration scenarios	Simulated	Source; screenshot of controls
Dashboard live mode (Supabase polling)	Implemented in source	Source; demonstrated on the prototype
ESP32 sensing and alert logic	Implemented in firmware	Hardware photo; firmware source
Supabase upload, Telegram messages	Implemented in firmware source	Firmware source; demonstrated on the prototype
Battery voltage / current	Not fitted	–
GPS, solar panel	Planned	–

5.6 Testing and Evidence

The tests below were designed from the implemented behaviour. The “Result / evidence” column states what supports each test: tests T1–T5 were executed on the supplied website, and tests T6–T12 concern the live hardware and are demonstrated on the prototype. No measured values have been invented.

Table 5.2: Test cases and their evidence status

ID	Test	Expected behaviour (from the implementation)	Result / evidence
T1	Search, category filter, sort	Catalogue list changes; model count shown (“8 models”)	Pass – phone screenshot and source execution
T2	Finder validation	Moving on without an answer shows “Choose one option to continue.”	Pass – source execution
T3	Ranking worked example	Top three: Yakuza Neu (100), Yakuza Viraj (100), Komaki X-One (96.4 → 96%)	Pass – source execution; see Table above
T4	Compare three models	Side-by-side cards for three models	Pass – source execution
T5	Dashboard with no hardware	Shows Offline, “No live data”, no invented values	Pass – phone screenshots
T6	Dashboard live mode	Row younger than 15 s → “ESP32 online”, values update every ~2 s	Demonstrated on the prototype
T7	Temperature warning/critical	Heating a DS18B20 above 55 °C (motor) or 45 °C (battery) → warning, buzzer 2 beeps, alert row	Demonstrated on the prototype
T8	Vibration and impact	Shaking raises vibration; large jolt (> 12) → IMPACT critical	Demonstrated on the prototype
T9	Parked movement	After 30 s still, moving or tilting > 30° → THEFT critical	Demonstrated on the prototype
T10	Telegram message	Message received for warning/critical and “cleared”	Demonstrated on the prototype
T11	Wi-Fi loss	Node keeps displaying and sounding alarms offline	Demonstrated on the prototype
T12	Offline detection	No row for more than 15 s → dashboard returns to Offline	Demonstrated on the prototype

5.7 Observations on the Current Implementation

While studying the supplied files, the following points were noted. They are listed so that they can be corrected or explained at the viva.

- **Inputs differ from the presentation.** The presentation mentions height and weight as inputs; the implemented finder uses five inputs (use, budget, range, terrain, priority).
- **Simulated quantities in live mode.** In the supplied source, state of charge and state of health stay at their simulator values even when live telemetry arrives, and battery temperature is not shown on the dashboard KPI cards (it is used by the device for alerts). The deployed build shows “no live values” when offline, so the team should confirm which version is final.
- **Units.** The firmware reports vibration in m/s² in alerts, while the dashboard column is `vibration_g` and the page displays g. The conversion happens in the unseen part of the firmware and should be confirmed.

- **Presentation items not implemented.** The presentation lists a smart BMS, an LCD, a current sensor, GPS and an optional LCD/mobile interface; of these only the OLED is fitted, and the current sensor, GPS and BMS are not part of the working prototype.
- **Credentials.** The prototype source contains a Wi-Fi password, a database key and a Telegram bot token. These are not reproduced in this report. Before the code is shared or deployed they must be removed from the source, the Telegram token regenerated and database access restricted with row-level security.
- **Character encoding.** Some icons in the supplied web page show garbled characters (for example in the "why this scooter" list); this is a text-encoding issue in the source and does not affect the scoring.

5.8 Overall Discussion

The implemented prototype demonstrates that an explainable recommendation stage and a sensor-based monitoring stage can be combined in one low-cost web platform. The recommendation stage is complete for the eight-model catalogue and behaves deterministically. The monitoring stage has the complete firmware logic for temperature, vibration, tilt, impact and parked-movement alerts, and the dashboard has a complete live-or-offline data path. The principal gaps are the absence of voltage and current measurement, the small and indicative catalogue, the fixed weights and the lack of road testing. These gaps are stated openly because a monitoring or recommendation system is only trustworthy when its limits are clear.

5.9 Summary

This chapter presented the recommendation results, the offline dashboard behaviour, the status of every feature, the functional test plan with its evidence status, the limitations observed in the current implementation and an overall discussion. The next chapter describes the live monitoring tool and the demonstration modes.

CHAPTER 6
LIVE MONITORING TOOL AND DEMONSTRATION

6.1 Overview

The live monitoring tool is the monitoring section of the VoltWise website. It presents the condition of the selected vehicle in one view: the connection state of the ESP32 node, six key indicators, extended telemetry, a trend chart, the movement panel and the alert centre. It operates in one of three modes: live hardware, offline, or demonstration, and always shows which mode is active.

6.2 Live Monitoring Dashboard

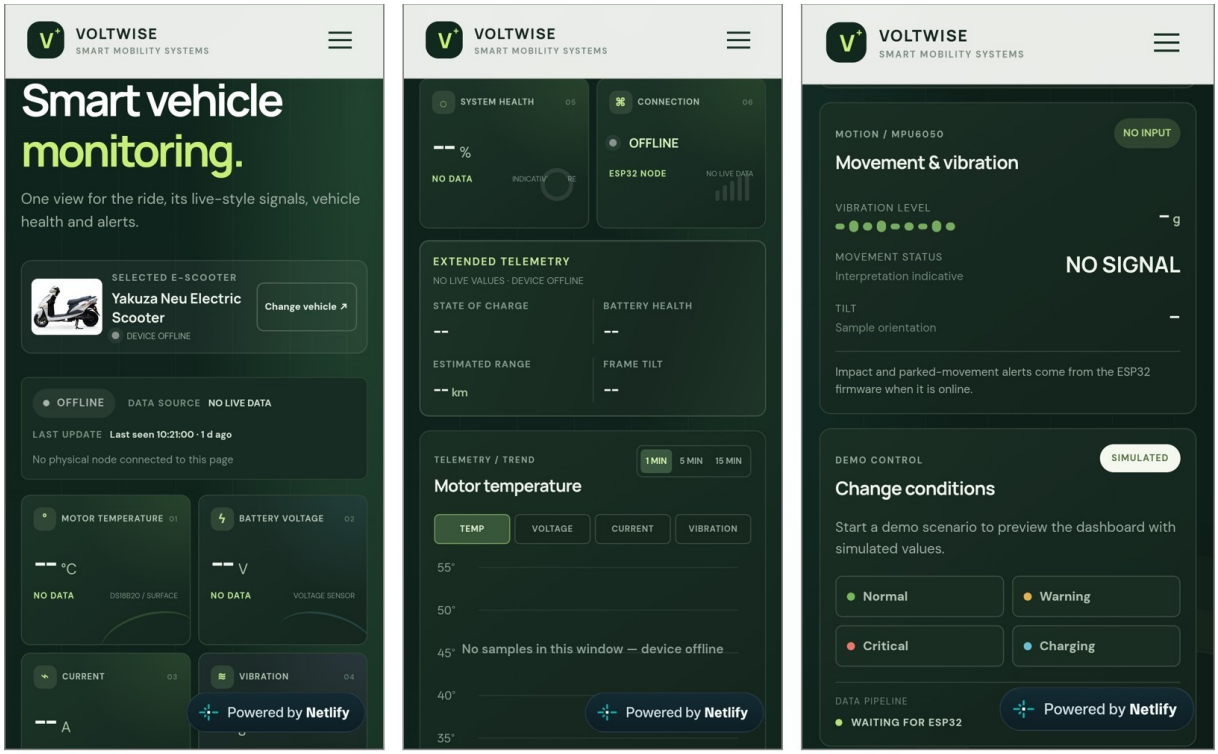
6.2.1 Monitoring Dashboard

The monitoring section starts with the selected e-scooter and the connection state. Six KPI cards show motor temperature (DS18B20 / surface), battery voltage, current, vibration (MPU6050), an indicative system-health score and the connection state of the ESP32 node. An "extended telemetry" strip shows state of charge, battery health, estimated range and frame tilt. A trend chart can show temperature, voltage, current or vibration over 1, 5 or 15 minutes. Further panels show a health ring, a battery graphic, a movement and vibration panel, a hardware-readiness panel, maintenance insights and the alert centre.

Live data versus simulated data. The page distinguishes three situations, and the report keeps the same distinction:

- **Hardware online:** the bridge script has received a row from Supabase that is younger than 15 s. The connection label changes to "ESP32 online / live hardware" and the status shown is the one decided by the device.
- **Hardware offline:** no row, or only an old row, exists. The label shows "Offline". In the deployed build captured in the screenshots, the cards show "No data", the chart reports that there are no samples in the window and the last-seen time is displayed.
- **Demonstration mode:** the "Change conditions" buttons (Normal, Warning, Critical, Charging) load fixed demonstration values. These are explicitly labelled "Simulated" and are not physical measurements. The Fleet and Analytics sections use 12 fictional units and session-only simulated samples.

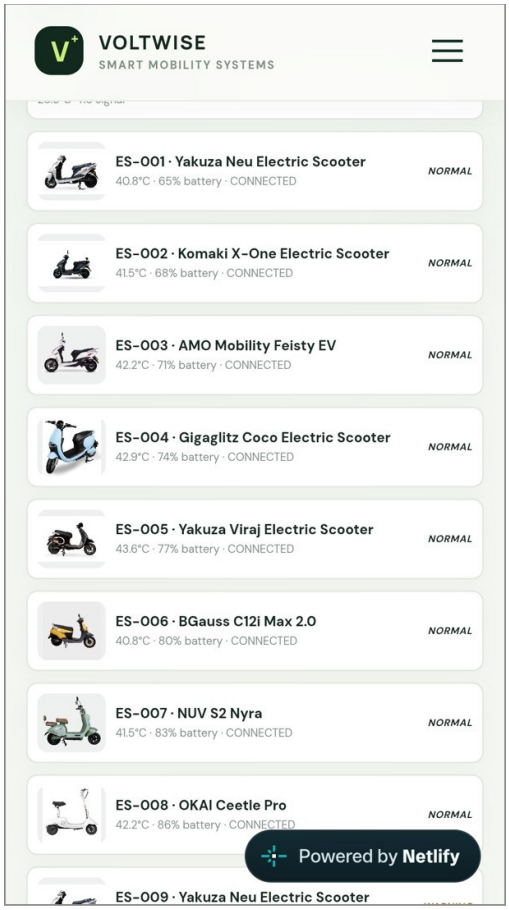
For the demonstration mode the dashboard applies its own status rule: CRITICAL if temperature is at least 50 °C or state of charge is at most 8%; WARNING if temperature is at least 44 °C, state of charge is at most 20% or state of health is below 75%; CHARGING if the charging flag is set; otherwise NORMAL. The "system health" percentage is an indicative score computed from temperature, state of charge, state of health and vibration; it is a formula and not a measured quantity. When the hardware is online, the status reported by the device replaces this rule so that the website and the device always agree.



(a) Monitor header **(b) Telemetry and chart** **(c) Motion and demo control**

Figure 6.1: Monitoring dashboard on a phone with the ESP32 node offline: (a) header, (b) extended telemetry and trend chart, (c) motion panel and demonstration control

The screenshots in Figure 6.1 were captured while the hardware was not connected. They show the dashboard reporting “Offline”, “No live data” and “No physical node connected to this page”, and the data pipeline waiting for the ESP32. They should be read as evidence that the page does not display invented values when no live row is available; they are not evidence of a hardware fault. The “Last seen” field in this build shows a time stamp from the previous day, which is consistent with a row having been received earlier, and the online connection is demonstrated on the prototype.



Simulated fleet list

Figure 6.2: Simulated fleet list on the Fleet section (fictional demonstration units)

6.3 Telemetry Demonstration

6.3.1 Dashboard Demonstration Modes

For presentations without hardware, the “Change conditions” buttons load fixed demonstration values: Normal (42.5 °C, 48.2 V, 6.4 A, 0.18 g, charge 78%), Warning (45.8 °C, 43.6 V, 8.2 A, 0.62 g, charge 18%), Critical (52.4 °C, 39.4 V, 11.8 A, 1.08 g, charge 7%) and Charging (39.6 °C, 52.1 V, 4.7 A, 0.12 g, charge 84%). These are simulated values and must be described as such. Figure 6.3 and Figure 6.4 show the Warning and Critical modes as rendered from the supplied source file.

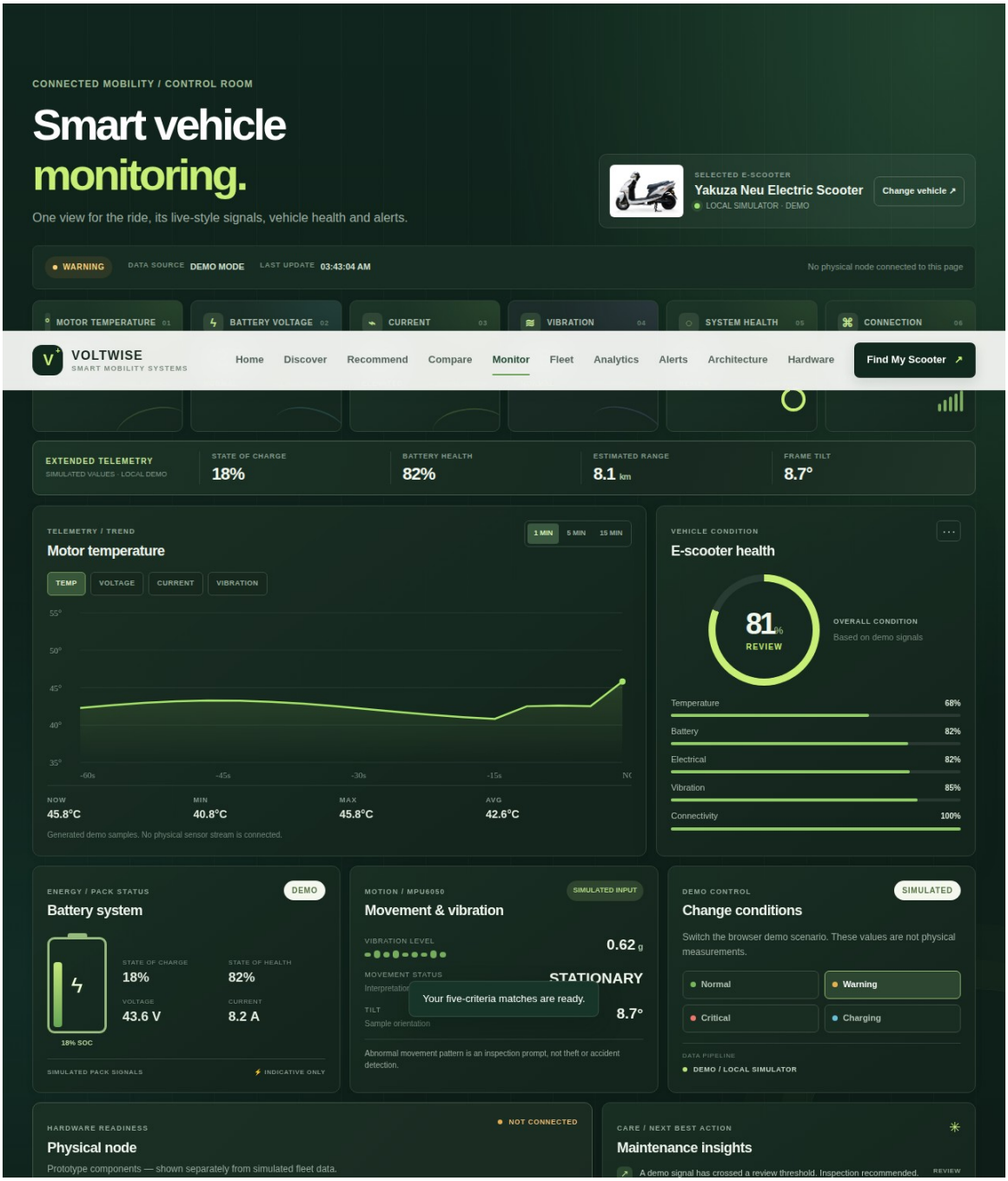


Figure 6.3: Dashboard in the Warning demonstration mode (simulated values, rendered from the source file)

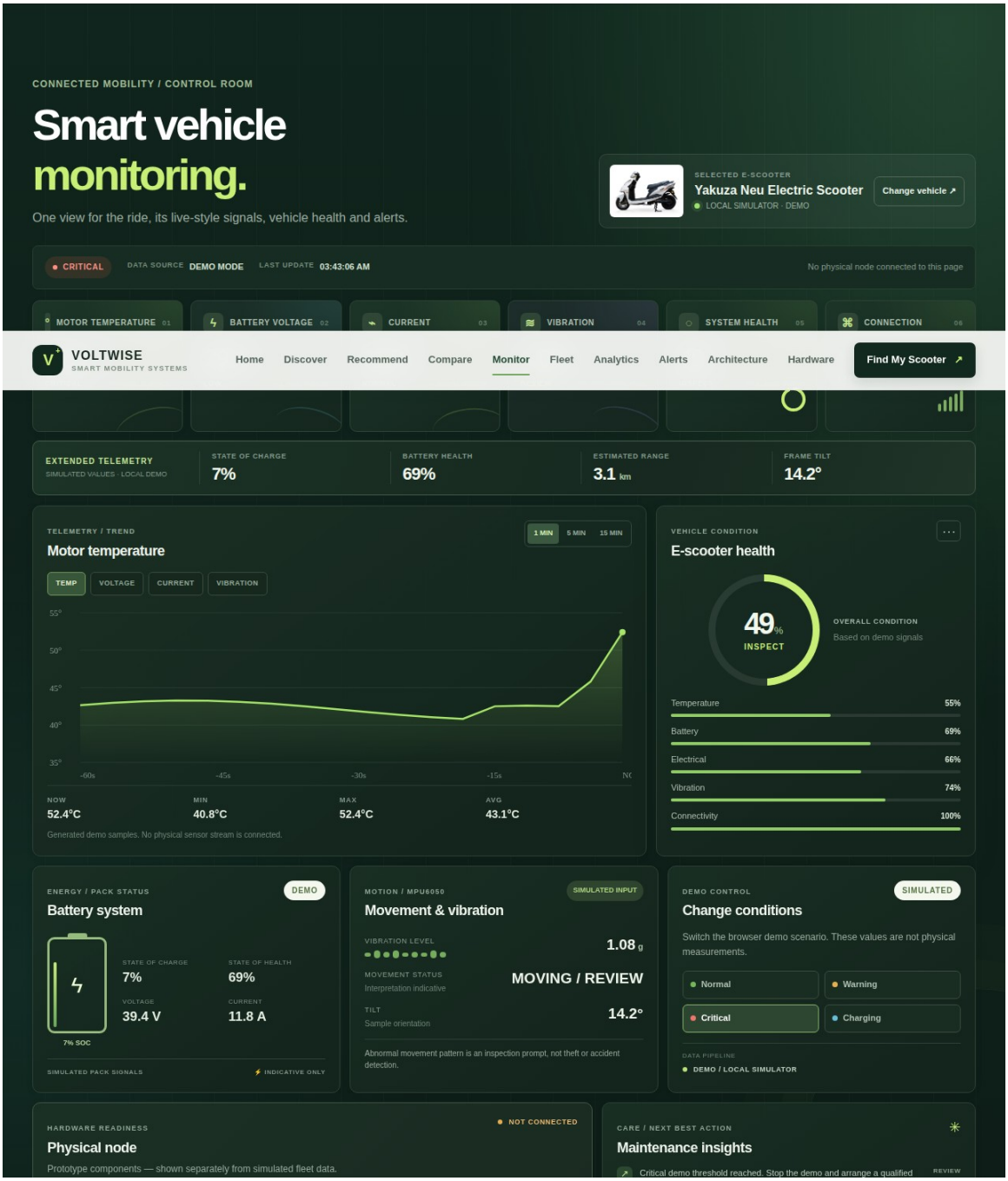


Figure 6.4: Dashboard in the Critical demonstration mode (simulated values, rendered from the source file)

6.4 Application Areas

- **Personal e-scooter and e-bike owners:** early warning of over-temperature, abnormal vibration, impact and movement while parked.
- **First-time buyers:** an explained shortlist in showrooms, colleges and online stores.
- **Campus and small-fleet operators:** a future multi-node view once real voltage, current and location sensing are added.

- **Education:** a teaching example of IoT sensing, cloud storage, rule-based decision support and honest data labelling.

6.5 Alerts and Telegram Messages

When the firmware raises a warning or critical alert it writes a row to the alert table and sends a Telegram message containing the device name, level, code and reason. The dashboard reads new alert rows and lists them in the alert centre with level, title, message and time; the first fetch after loading only records the newest alert identifier so that old alerts are not replayed. Alerts can be filtered by level and cleared, and an optional tone can be switched on or off in the browser.

6.6 Summary

This chapter described the live monitoring tool, its three data modes, the demonstration scenarios and the alert path. The demonstration values are simulated and are labelled so on the page; only values received from the ESP32 node through Supabase are live readings.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project built VoltWise, a web platform that joins an explainable e-scooter recommendation stage with a sensor-based monitoring stage. The website lists eight multi-brand models, supports search, filtering, sorting, comparison and shortlisting, and ranks the models from five rider answers using a transparent weighted score (use 35%, budget 25%, range 15%, terrain 15%, priority 10%). The score is rule-based, not a trained model, and the page says so. Running the supplied code with sample answers produced ranked top-three results with match percentages and reasons.

The monitoring stage is an ESP32 prototype with two DS18B20 sensors, an MPU6050, a BMP280, an OLED, a buzzer and two LEDs. Its firmware learns the mounting orientation, derives vibration and tilt, applies confirmation and hysteresis, classifies NORMAL, WARNING and CRITICAL states, alarms locally, and uploads telemetry and alerts to Supabase and Telegram. The dashboard polls Supabase and shows an honest offline state, as captured in the screenshots, when no recent data exists.

The work was presented as a research paper titled “AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System” at the AICTE-sponsored International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026), organised by the Institute of CSE, SIMATS Engineering, Thandalam, Chennai, on 21 and 22 October 2026. The participation certificates of the three team members are included at the end of this report.

7.2 Achievement of Project Objectives

Table 7.1: Objectives and their achievement

No.	Objective	Status
1	Multi-brand catalogue with search, filter, sort	Achieved (8 models, indicative data)
2	Five-criterion weighted scoring with fixed weights	Achieved (rule-based)
3	Top-three results with match percentage and reasons	Achieved
4	Comparison of two or three models and shortlist	Achieved
5	ESP32 firmware for temperature, vibration, tilt, ambient reading and orientation calibration	Achieved in firmware
6	WARNING/CRITICAL logic with confirmation and hysteresis, impact and parked-movement alerts	Achieved in firmware
7	Telemetry and alert upload to Supabase and Telegram messages	Implemented in firmware
8	Dashboard with live, offline and demonstration modes	Achieved; offline state evidenced by screenshots

7.3 Limitations

- The catalogue contains eight models with indicative data that must be updated manually.
- Scoring weights are fixed and the method does not learn from user feedback.
- Voltage, current, state of charge and state of health are not measured by the present hardware.
- The prototype is on a breadboard and has not been tested on a vehicle or on the road.
- Thresholds are engineering estimates and have not been calibrated.
- Recorded live-run measurements are not included in this report.

7.4 Future Scope

- Fit the INA219 and measure real battery voltage and current; derive state of charge from measurements.
- Add the NEO-6M GPS module for location and theft tracking, and the solar panel for charging studies.
- Mount the node on a real vehicle, record data over rides and calibrate temperature, vibration and tilt thresholds.
- Enlarge the catalogue, keep specifications up to date from manufacturer sources and let users adjust criterion weights (for example with AHP).
- Collect user feedback and, only when enough data exists, evaluate a learning-based ranking against the rule-based baseline.
- Add user accounts, row-level security, encrypted credential storage and an installable mobile app.
- Add predictive maintenance using logged temperature and vibration history, and a multi-node fleet view with real data.

7.5 Summary

The project achieved its main objective of demonstrating an integrated, transparent recommendation and monitoring platform for electric two-wheelers. The limitations listed above define the work needed to move from a prototype to a field-ready system.

REFERENCES

[1] S. Yan, N. E. O'Connor and M. Liu, "U-Park: A user-centric smart parking recommendation system for electric shared micromobility services," IEEE Trans. Artif. Intell., vol. 5, no. 10, pp. 5179–5193, Oct. 2024.

[2] J. D. Flores Mendieta, V. E. Iñiguez Moran, J. M. Guaman Ochoa, D. V. Ochoa Correa and D. F. Astudillo Salinas, "An IoT-based measurement device for e-bike tracking performance analysis," in Proc. 2024 IEEE 8th Ecuador Tech. Chapters Meeting (ETCM), Cuenca, Ecuador, Oct. 2024, pp. 1–6.

[3] S. Umathe and R. Hiware, "Artificial intelligence and IoT based smart battery management system for electric vehicle," in Proc. 2022 Int. Conf. Smart Generation Computing, Communication and Networking (SMART GENCON), IEEE, 2022, pp. 1–7.

[4] I. Ali, M. R. Hazari and M. A. Mannan, "IoT-based smart battery management and monitoring system for electric vehicles," AIUB J. Sci. Eng., vol. 22, no. 2, pp. 181–188, Aug. 2023.

[5] Y. Zhang and X. Chen, "Explainable recommendation: A survey and new perspectives," Found. Trends Inf. Retr., vol. 14, no. 1, pp. 1–101, 2020.

[6] C.-L. Hwang and K. Yoon, Multiple Attribute Decision Making: Methods and Applications – A State-of-the-Art Survey. Berlin, Germany: Springer, 1981.

[7] T. L. Saaty, The Analytic Hierarchy Process. New York, NY, USA: McGraw-Hill, 1980.

[8] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," IEEE Trans. Knowl. Data Eng., vol. 17, no. 6, pp. 734–749, Jun. 2005.

[9] F. Ricci, L. Rokach and B. Shapira, Eds., Recommender Systems Handbook, 2nd ed. New York, NY, USA: Springer, 2015.

[10] J. Gubbi, R. Buyya, S. Marusic and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," Future Gener. Comput. Syst., vol. 29, no. 7, pp. 1645–1660, 2013.

[11] A. Al-Fuqaha et al., "Internet of Things: A survey on enabling technologies, protocols, and applications," IEEE Commun. Surveys Tuts., vol. 17, no. 4, pp. 2347–2376, 4th Quart. 2015.

[12] Maxim Integrated, "DS18B20 programmable resolution 1-Wire digital thermometer," data sheet.

[13] InvenSense, "MPU-6000 and MPU-6050 product specification," Rev. 3.4, 2013.

[14] Bosch Sensortec, "BMP280 digital pressure sensor," data sheet.

[15] Espressif Systems, "ESP32 series data sheet."

[16] Supabase, "Supabase documentation – REST API." [Online]. Available: https://supabase.com/docs

[17] Telegram, "Telegram Bot API." [Online]. Available: https://core.telegram.org/bots/api

APPENDIX

9.1 Telemetry and Configuration Variables

Table 9.1: Variables used by the monitoring node and dashboard

Variable	Meaning	Unit / values	Source
motorTemp / motor_temp	Motor surface temperature	°C	DS18B20 index 0
batteryTemp / battery_temp	Battery surface temperature	°C	DS18B20 index 1
ambientTemp	Ambient temperature	°C	BMP280
pressure	Air pressure	hPa	BMP280
vibration / vibration_g	Length of the difference between current and resting acceleration	m/s² in firmware alerts; shown as g in the dashboard	MPU6050
tiltAngle / tilt	Angle between current and reference gravity vector	degrees	MPU6050
impactDetected	Vibration above 12.0, latched 8 s	true / false	Derived
voltage, current	Battery voltage and current	V, A (mA in firmware)	INA219 – not fitted
status	NORMAL, WARNING or CRITICAL	state	Derived
code, reason	Alert code and readable reason	text	Derived
DEVICE_ID	Identifier of the node	ebike-01	Configuration

9.2 Alert Rule Conditions

Table 9.2: Alert rules implemented in the firmware

Rule	Condition	Level	Response
R1	Motor temperature > 55 °C (clear < 52 °C), two samples	Warning	Buzzer 1200 Hz ×2; alert upload; Telegram
R2	Motor temperature > 70 °C (clear < 66 °C)	Critical	Buzzer 2000 Hz ×5
R3	Battery temperature > 45 °C (clear < 42 °C)	Warning	As R1
R4	Battery temperature > 55 °C (clear < 51 °C)	Critical	As R2
R5	Vibration > 2.5 (clear < 1.8)	Warning	As R1
R6	Vibration > 6.0 (clear < 4.5)	Critical	As R2
R7	Parked (still 30 s) and vibration > 2.5 or tilt > 30°, two samples	Critical	Buzzer 2200 Hz ×6
R8	Vibration > 12.0	Critical	Buzzer 2600 Hz ×8

SOURCE CODE

10.1 Overview of the Source Code

The project consists of two programs: the ESP32 firmware (C++ with the Arduino framework) and the single-file website (HTML, CSS and JavaScript). The listings below reproduce the main parts. Wi-Fi, database and Telegram credentials have been replaced by placeholders and embedded image data by the word DATAURI. The complete website file, including the recommendation engine and the dashboard script, is in the project archive.

10.2 ESP32 Firmware (excerpts)

```
// ===== Excerpt 1: configuration, pins, timing, thresholds (secrets removed) =====
#define HAS_INA219   false      // true once the INA219 is wired in
#define USE_SUPABASE true
#define USE_TELEGRAM true

const char* WIFI_SSID        = "<WIFI_SSID>";          // removed from report
const char* WIFI_PASS        = "<WIFI_PASSWORD>";      // removed from report
const char* SUPABASE_URL     = "<SUPABASE_PROJECT_URL>";
const char* SUPABASE_KEY     = "<SUPABASE_PUBLIC_KEY>";
const char* DEVICE_ID        = "ebike-01";
const char* TELEGRAM_TOKEN   = "<TELEGRAM_BOT_TOKEN>"; // removed from report
const char* TELEGRAM_CHAT_ID = "<TELEGRAM_CHAT_ID>";

#define PIN_ONEWIRE     4
#define PIN_BUZZER      5
#define PIN_LED_RED    25
#define PIN_LED_GREEN  26
// I2C on the default GPIO21 (SDA) / GPIO22 (SCL)

const unsigned long READ_INTERVAL     = 500;    // sensor + display refresh
const unsigned long SUPABASE_INTERVAL = 3000;   // one telemetry row
const unsigned long TELEGRAM_COOLDOWN = 60000;  // min gap for the same code

// Each alert has a trigger value and a lower clear value (hysteresis)
float TEMP_MOTOR_WARN  = 55.0, TEMP_MOTOR_WARN_CLR = 52.0;
float TEMP_MOTOR_CRIT  = 70.0, TEMP_MOTOR_CRIT_CLR = 66.0;
float TEMP_BATT_WARN   = 45.0, TEMP_BATT_WARN_CLR  = 42.0;
float TEMP_BATT_CRIT   = 55.0, TEMP_BATT_CRIT_CLR  = 51.0;
float VIBRATION_WARN   = 2.5,  VIBRATION_WARN_CLR  = 1.8;
float VIBRATION_CRIT   = 6.0,  VIBRATION_CRIT_CLR  = 4.5;
float IMPACT_THRESHOLD = 12.0;
float TILT_THRESHOLD   = 30.0;   // degrees away from the mounted position
float IDLE_VIBRATION   = 0.8;
unsigned long ARM_DELAY = 30000; // stillness before parking mode arms
const uint8_t CONFIRM_SAMPLES = 2;          // reads a condition must hold
const unsigned long IMPACT_LATCH_MS = 8000; // impact stays visible this long

enum Status { NORMAL = 0, WARNING = 1, CRITICAL = 2 };

// ===== Excerpt 2: orientation calibration =====
void calibrateOrientation() {
  float sx = 0, sy = 0, sz = 0, sMag = 0;
  sensors_event_t a, g, t;
  for (int i = 0; i < 100; i++) {
    mpu.getEvent(&a, &g, &t);
    sx += a.acceleration.x;  sy += a.acceleration.y;  sz += a.acceleration.z;
    sMag += sqrt(a.acceleration.x * a.acceleration.x +
                 a.acceleration.y * a.acceleration.y +
                 a.acceleration.z * a.acceleration.z);
    delay(8);
```

```
  }
  refX = sx / 100.0; refY = sy / 100.0; refZ = sz / 100.0;
  restingAccel = sMag / 100.0;
  // ... serial report of the learned mounting (flat / vertical) ...
}

// ===== Excerpt 3: reading the sensors =====
void readSensors() {
  sensors.requestTemperatures();
  r.motorTemp   = sensors.getTempC(addrMotor);
  r.batteryTemp = sensors.getTempC(addrBattery);

  sensors_event_t a, g, t;
  mpu.getEvent(&a, &g, &t);
  float ax = a.acceleration.x, ay = a.acceleration.y, az = a.acceleration.z;

  // Vibration = magnitude of the DIFFERENCE from the resting vector
  float dx = ax - refX, dy = ay - refY, dz = az - refZ;
  r.vibration = sqrt(dx * dx + dy * dy + dz * dz);

  // Tilt = angle between current gravity vector and the boot reference
  float magNow = sqrt(ax * ax + ay * ay + az * az);
  float magRef = sqrt(refX * refX + refY * refY + refZ * refZ);
  if (magNow > 0.1 && magRef > 0.1) {
    float dot = (ax * refX + ay * refY + az * refZ) / (magNow * magRef);
    dot = constrain(dot, -1.0, 1.0);
    r.tiltAngle = acos(dot) * 180.0 / PI;
  } else {
    r.tiltAngle = 0;
  }

  r.impactDetected = (r.vibration > IMPACT_THRESHOLD);
  if (r.impactDetected) impactUntil = millis() + IMPACT_LATCH_MS;

  r.ambientTemp = bmp.readTemperature();
  r.pressure    = bmp.readPressure() / 100.0F;

#if HAS_INA219
  r.voltage = ina219.getBusVoltage_V();
  r.current = ina219.getCurrent_mA();
#else
  r.voltage = NAN;
  r.current = NAN;
#endif
  r.timestamp = millis();
}

// ===== Excerpt 4: parking state, confirmation and hysteresis =====
void updateParkingState() {
  if (r.vibration < IDLE_VIBRATION) {
    if (!parkingArmed && millis() - stillSince > ARM_DELAY) {
      parkingArmed = true;
      Serial.println("[PARKING] armed - vehicle has been still");
    }
  } else {
    stillSince = millis();
  }
}

// Confirm over several reads, clear at a lower value than it triggers.
bool confirm(bool over, bool under, uint8_t &counter, bool &flag) {
  if (!flag) {
    if (over) { if (++counter >= CONFIRM_SAMPLES) { flag = true; counter = 0; } }
    else counter = 0;
  } else {
    if (under) { if (++counter >= CONFIRM_SAMPLES) { flag = false; counter = 0; } }
```

```
    else counter = 0;
  }
  return flag;
}

// ===== Excerpt 5: status evaluation =====
void evaluateStatus() {
  confirm(r.motorTemp   > TEMP_MOTOR_WARN, r.motorTemp   < TEMP_MOTOR_WARN_CLR, cMotorW, fMotorW);
  confirm(r.motorTemp   > TEMP_MOTOR_CRIT, r.motorTemp   < TEMP_MOTOR_CRIT_CLR, cMotorC, fMotorC);
  confirm(r.batteryTemp > TEMP_BATT_WARN,  r.batteryTemp < TEMP_BATT_WARN_CLR,  cBattW,  fBattW);
  confirm(r.batteryTemp > TEMP_BATT_CRIT,  r.batteryTemp < TEMP_BATT_CRIT_CLR,  cBattC,  fBattC);
  confirm(r.vibration   > VIBRATION_WARN,  r.vibration   < VIBRATION_WARN_CLR,  cVibW,   fVibW);
  confirm(r.vibration   > VIBRATION_CRIT,  r.vibration   < VIBRATION_CRIT_CLR,  cVibC,   fVibC);

  bool parkMove = parkingArmed &&
                  (r.vibration > VIBRATION_WARN || r.tiltAngle > TILT_THRESHOLD);
  if (parkMove) { if (cParkMove < 255) cParkMove++; } else cParkMove = 0;
  bool theft  = (cParkMove >= CONFIRM_SAMPLES);
  bool impact = (millis() < impactUntil);

  Status s = NORMAL;
  String why = "All systems normal", code = "OK";

  if (fMotorW) { s = WARNING; why = "Motor temperature elevated";   code = "MOTOR_TEMP_WARN"; }
  if (fBattW)  { s = WARNING; why = "Battery temperature elevated"; code = "BATT_TEMP_WARN"; }
  if (fVibW)   { s = WARNING; why = "Abnormal vibration";           code = "VIB_WARN"; }
  if (fMotorC) { s = CRITICAL; why = "Motor overheating";   code = "MOTOR_TEMP_CRIT"; }
  if (fBattC)  { s = CRITICAL; why = "Battery overheating"; code = "BATT_TEMP_CRIT"; }
  if (fVibC)   { s = CRITICAL; why = "Severe vibration";    code = "VIB_CRIT"; }
  if (theft)   { s = CRITICAL; why = "Movement while parked"; code = "THEFT"; }
  if (impact)  { s = CRITICAL; why = "Impact detected";       code = "IMPACT"; }

  r.status = s; r.reason = why; r.code = code;

  // Act once on entry, not on every cycle.
  if (s != prevStatus) {
    if (s == CRITICAL) {
      critCount++;
      if      (code == "IMPACT") startBuzzer(2600, 90, 60, 8);
      else if (code == "THEFT")  startBuzzer(2200, 250, 120, 6);
      else                       startBuzzer(2000, 400, 200, 5);
      alertScreenUntil = millis() + 4000;
      pending = {true, s, code, why};      // sent after the buzzer finishes
    } else if (s == WARNING) {
      warnCount++;
      startBuzzer(1200, 120, 400, 2);
      alertScreenUntil = millis() + 2500;
      pending = {true, s, code, why};
    }
    // ... return-to-normal handling continues in the full sketch ...
  }
}

// ===== Excerpt 6: dispatch of a queued alert from loop() =====
if (pending.active && buzzerIdle()) {
  Status lvl = pending.level;  String cd = pending.code;  String rsn = pending.reason;
  pending.active = false;
  if (lvl == NORMAL) {
#if USE_TELEGRAM
    telegramSend(String("Cleared - ") + DEVICE_ID + " is back to normal.");
#endif
  } else {
#if USE_SUPABASE
    uploadAlert(lvl, cd, rsn);
#endif
#if USE_TELEGRAM
```

```
    telegramAlert(lvl, cd, rsn);
#endif
  }
}
```

10.3 Website Recommendation and Bridge Logic (excerpts)

```
// ===== Excerpt 1: weights and the five criterion functions (voltwise.html) =====
const weights = {use:35, budget:25, range:15, terrain:15, priority:10};

function scoreUse(b, a) {
  if (b.use.includes(a)) return 100;
  return a === 'City / Commuting' && (b.category === 'COMMUTER' || b.category === 'CITY')
    ? 90 : a === 'Mixed Use' ? 70 : 35;
}
function scoreBudget(b, a) {
  const band = b.price < 60000 ? 0 : b.price < 90000 ? 1 : b.price <= 120000 ? 2 : 3;
  const chosen = ['Under ₹60,000','₹60,000–₹90,000','₹90,000–₹1,20,000','Above
₹1,20,000'].indexOf(a);
  return [100, 74, 48, 24][Math.abs(band - chosen)];
}
function scoreRange(b, a) {
  const band = b.range < 40 ? 0 : b.range < 60 ? 1 : b.range <= 80 ? 2 : 3;
  const chosen = ['Under 40 km','40–60 km','60–80 km','80+ km'].indexOf(a);
  return [100, 76, 50, 25][Math.abs(band - chosen)];
}
function scoreTerrain(b, a) {
  if (b.terrain.includes(a)) return 100;
  return b.terrain.includes('Mixed') ? 72 : 38;
}
function scorePriority(b, a) {
  if (b.priority.includes(a)) return 100;
  if (a === 'Balanced') return 78;
  if (a === 'Range') return Math.min(100, Math.round(b.range / 90 * 100));
  return 48;
}

// ===== Excerpt 2: weighted score and ranking =====
function scoreBike(b) {
  const raw = {
    use: scoreUse(b, state.answers.use),
    budget: scoreBudget(b, state.answers.budget),
    range: scoreRange(b, state.answers.range),
    terrain: scoreTerrain(b, state.answers.terrain),
    priority: scorePriority(b, state.answers.priority)
  };
  const points = Object.fromEntries(
    Object.entries(raw).map(([k, v]) => [k, v * weights[k] / 100]));
  return {...b, raw, points,
          score: Number(Object.values(points).reduce((a, v) => a + v, 0).toFixed(1))};
}
// in runWizard(), after the fifth answer:
state.ranked = bikes.map(scoreBike).sort((a, b) => b.score - a.score || a.price - b.price);
// renderResults() shows state.ranked.slice(0, 3)

// ===== Excerpt 3: dashboard status rule and health score (demonstration logic) =====
function getStatus(t = state.telemetry) {
  if (t.temperature >= 50 || t.soc <= 8) return 'CRITICAL';
  if (t.temperature >= 44 || t.soc <= 20 || t.soh < 75) return 'WARNING';
  if (t.charging) return 'CHARGING';
  return 'NORMAL';
}
function healthScore() {
  const t = state.telemetry;
  return Math.max(34, Math.min(100, Math.round(100
    - Math.max(0, t.temperature - 40) * 1.1
```

```
    - Math.max(0, 30 - t.soc) * 0.7
    - Math.max(0, 85 - t.soh) * 0.7
    - Math.max(0, t.vibration - 0.5) * 18)));
}

// ===== Excerpt 4: Supabase bridge (second script block; key removed) =====
const SUPABASE_URL = '<SUPABASE_PROJECT_URL>';
const SUPABASE_KEY = '<SUPABASE_PUBLIC_KEY>';
const DEVICE_ID = 'ebike-01';
const POLL_MS = 2000;
const STALE_MS = 15000;   // no new row for this long = device offline

async function poll() {
  try {
    const res = await api(`ebike_telemetry?device_id=eq.${encodeURIComponent(DEVICE_ID)}` +
                          `&order=created_at.desc&limit=1`);
    if (!res.ok) throw new Error('HTTP ' + res.status);
    const rows = await res.json();
    if (!rows.length) { goOffline('no rows for ' + DEVICE_ID + ' yet.'); return; }
    const row = rows[0];
    const age = Date.now() - new Date(row.created_at).getTime();
    if (age > STALE_MS) goOffline('device not reporting.');
    else applyRow(row);
    fetchAlerts();
  } catch (err) { /* two consecutive failures => offline */ }
}

function applyRow(row) {
  const payload = {
    temperature: num(row.motor_temp),
    vibration: num(row.vibration_g),
    tilt: num(row.tilt),
    charging: false,
    source: 'SUPABASE'
  };
  // voltage/current exist only once the INA219 is fitted
  if (row.voltage !== null && row.voltage !== undefined) payload.voltage = num(row.voltage);
  if (row.current_a !== null && row.current_a !== undefined) payload.current = num(row.current_a);
  window.updateTelemetry(payload);
  state.demo = 'hardware';   // stop the simulator overwriting live values
}
setInterval(poll, POLL_MS);
```

10.4 Complete Supabase Bridge Script

```
/* Voltwise - Supabase backend bridge.
 *
 * The ESP32 writes rows to Supabase; this file reads them and feeds the
 * dashboard through the existing window.updateTelemetry() contract.
 *
 * DESIGN NOTE: this file adds NO elements and NO styles to the page. It
 * only writes into panels the site already has, so the layout and look are
 * unchanged. Connection state is reported in the browser console and in the
 * existing connection label, nowhere else.
 */
 (function () {
     const SUPABASE_URL = '<SUPABASE_PROJECT_URL>';
     const SUPABASE_KEY = '<SUPABASE_PUBLIC_KEY>';
     const DEVICE_ID    = 'ebike-01';
      const POLL_MS  = 2000;
     const STALE_MS = 15000;
     // no new row for this long = device offline
     const link = {
         connected: false,     lastRowId: null,     lastAlertId: null,     lastStatus: null,     failures:
0,     announced: false
  };
```

```
    const headers = {
       apikey: SUPABASE_KEY,     Authorization: 'Bearer ' + SUPABASE_KEY,     Accept: 'application/json'
};
    const api = (path) =>     fetch(SUPABASE_URL + '/rest/v1/' + path, {
   headers, cache: 'no-store'
});
    const num = (v) => (Number.isFinite(Number(v)) ? Number(v) : 0);
    function setText(sel, text) {
       const el = document.querySelector(sel);
       if (el) el.textContent = text;
}
    // The page's own renderMonitor() hardcodes the connection labels to
  // "CONNECTED" / "DEMO ONLINE" on every render, which would overwrite
  // anything set here and leave the dashboard claiming the ESP32 is online
  // when it is not. Wrapping renderMonitor lets the real state win, every
  // time the panel redraws.
  function applyConnectionLabels() {
       if (link.connected) {
           setText('#kpi-connection', 'ESP32 ONLINE');
           setText('#data-mode', 'LIVE HARDWARE');
           setText('#connection-label', 'ESP32 \u00B7 LIVE DATA');
  } else {
           setText('#kpi-connection', 'OFFLINE');
           setText('#data-mode', 'DEMO MODE');
           setText('#connection-label', 'LOCAL SIMULATOR \u00B7 DEMO');
  }
}
    function hookRenderMonitor() {
       if (hookRenderMonitor.done) return;
       if (typeof window.renderMonitor !== 'function') return;
       const base = window.renderMonitor;
       window.renderMonitor = function () {
           const out = base.apply(this, arguments);
           applyConnectionLabels();
           return out;
  };
       hookRenderMonitor.done = true;
}
    // While hardware is connected the DEVICE decides the status, not the
  // dashboard's own demo thresholds - otherwise the page can show WARNING
  // while the ESP32's OLED and LEDs say NORMAL.
  function installStatusOverride() {
       if (installStatusOverride.done) return;
       if (typeof window.getStatus !== 'function') return;
       const original = window.getStatus;
       window.getStatus = function (t) {
           if (link.connected && link.lastStatus) return link.lastStatus;
           return original(t);
  };
       installStatusOverride.done = true;
}
    async function poll() {
       try {
           const res = await api(         `ebike_telemetry?device_id=eq.${encodeURIComponent(DEVICE_ID)}`
+         `&order=created_at.desc&limit=1`       );
           if (!res.ok) throw new Error('HTTP ' + res.status);
           const rows = await res.json();
           link.failures = 0;
            if (!rows.length) {
              goOffline('Connected to Supabase, but no rows for ' + DEVICE_ID + ' yet.');
              return;
    }
            const row = rows[0];
           const age = Date.now() - new Date(row.created_at).getTime();
            if (age > STALE_MS) {
              goOffline('Last reading was ' + Math.round(age / 1000) + 's ago - device not reporting.');
```

```
     } else {
                applyRow(row);
     }
              link.lastRowId = row.id;
             fetchAlerts();
   } catch (err) {
             link.failures++;
             if (link.failures >= 2) goOffline('Could not reach Supabase: ' + err.message);
   }
 }
     function goOffline(why) {
        if (link.connected || !link.announced) {
             console.warn('[Voltwise] hardware offline -', why);
             link.announced = true;
   }
         link.connected = false;
         // Hand the page back to its own simulator so the dashboard keeps
        // working during a demo even if the device or network drops.
        if (typeof state === 'object' && state && state.demo === 'hardware') {
             state.demo = 'normal';
   }
         applyConnectionLabels();
 }
     function applyRow(row) {
        link.lastStatus = row.status || null;
        installStatusOverride();
        hookRenderMonitor();
         if (!link.connected) {
             link.connected = true;
             console.log('[Voltwise] live telemetry from Supabase -', DEVICE_ID);
             if (typeof toast === 'function') toast('Live telemetry connected.');
   }
          const payload = {
             temperature: num(row.motor_temp),       vibration: num(row.vibration_g),       tilt:
num(row.tilt),       charging: false,       source: 'SUPABASE'
    };
         // Voltage and current only exist once the INA219 is fitted. Leaving
        // them out keeps those panels on their last value rather than showing
        // a fabricated number.
        if (row.voltage   !== null && row.voltage   !== undefined) payload.voltage = num(row.voltage);
         if (row.current_a !== null && row.current_a !== undefined) payload.current = num(row.current_a);
          if (typeof window.updateTelemetry === 'function') window.updateTelemetry(payload);
          // Stop the built-in simulator overwriting live values.
        if (typeof state === 'object' && state) state.demo = 'hardware';
          // Only existing labels are updated - no new elements.
        applyConnectionLabels();
 }
     // ---- Alerts the device wrote ----
   async function fetchAlerts() {
         try {
             const res = await api(         `ebike_alerts?device_id=eq.${encodeURIComponent(DEVICE_ID)}` +
`&order=created_at.desc&limit=8`       );
             if (!res.ok) return;
             const rows = await res.json();
              // The first successful fetch only records where we are, so a reload
            // does not replay old alerts. This primes even on an empty table -
            // otherwise the first alert the device ever raises would be treated
            // as history and swallowed.
            if (link.lastAlertId === null) {
                 link.lastAlertId = rows.length ? rows[0].id : 0;
                 return;
     }
             if (!rows.length) return;
              for (const a of rows.filter(x => x.id > link.lastAlertId).reverse()) {
                 if (typeof addAlert === 'function') {
```

```
                    addAlert(a.level === 'CRITICAL' ? 'CRITICAL'                 : a.level === 'WARNING'
? 'WARNING' : 'INFO',                    a.reason || a.code,                   describe(a),
'sb-alert-' + a.id);
        }
                link.lastAlertId = a.id;
      }
    } catch (err) {
       /* alerts are best-effort */
    }
  }
      function describe(a) {
         const bits = [];
         if (a.motor_temp    != null) bits.push(`motor ${num(a.motor_temp).toFixed(1)}°C`);
         if (a.battery_temp  != null) bits.push(`battery ${num(a.battery_temp).toFixed(1)}°C`);
         if (a.vibration_ms2 != null) bits.push(`vibration ${num(a.vibration_ms2).toFixed(2)} m/s²`);
         if (a.tilt          != null) bits.push(`tilt ${num(a.tilt).toFixed(0)}°`);
         const when = new Date(a.created_at).toLocaleTimeString();
         return `Recorded by ${a.device_id} at ${when}. ${bits.join(', ')}.`;
  }
      document.addEventListener('DOMContentLoaded', () => {
         hookRenderMonitor();
         applyConnectionLabels();
         poll();
         setInterval(poll, POLL_MS);
  });
})();
```

CONFERENCE PARTICIPATION CERTIFICATES

Certificates of participation awarded to the team members for presenting the research paper "AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System" at ICAI-CEME 2026, SIMATS Engineering, Thandalam, Chennai (21–22 October 2026).



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

M. Soundarya

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate of M. Soundarya



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

MoE's INNOVATION CELL (GOVERNMENT OF INDIA)

SUSTAINABLE DEVELOPMENT GOALS

CERTIFICATE OF PARTICIPATION

This certificate is presented to

A. SARANIYA

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate of A. Saraniya



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

MoE's INNOVATION CELL (GOVERNMENT OF INDIA)

SUSTAINABLE DEVELOPMENT GOALS

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Josphin Sheran J

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

AI-Powered Multi-Brand E-Bike Recommendation and Smart Monitoring System

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate of Josphin Sheran J